[ADS5034-01] Computer Vision

11. Classification (1)

Jae-Young Choi

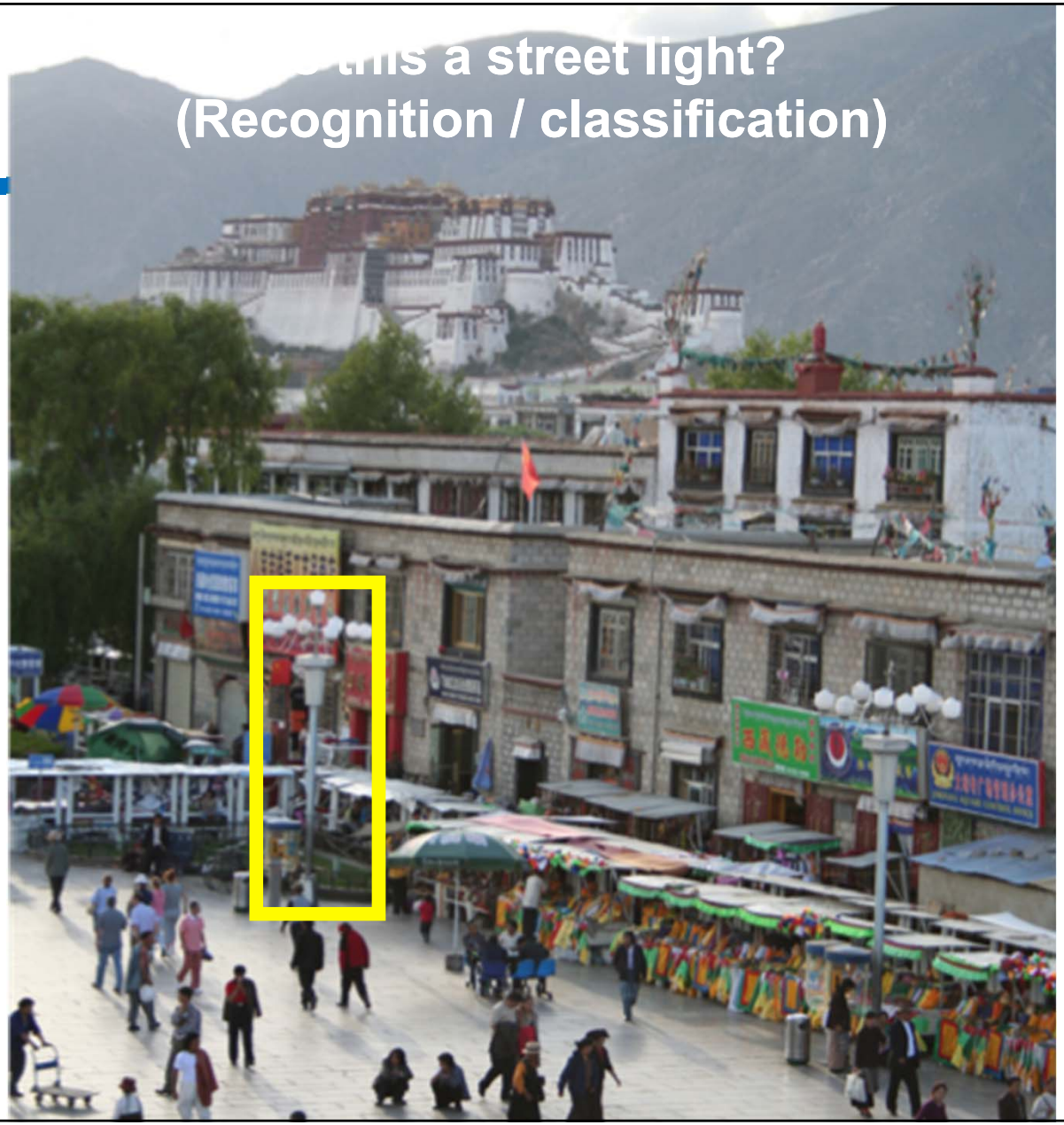
Sungkyunkwan University

Outline

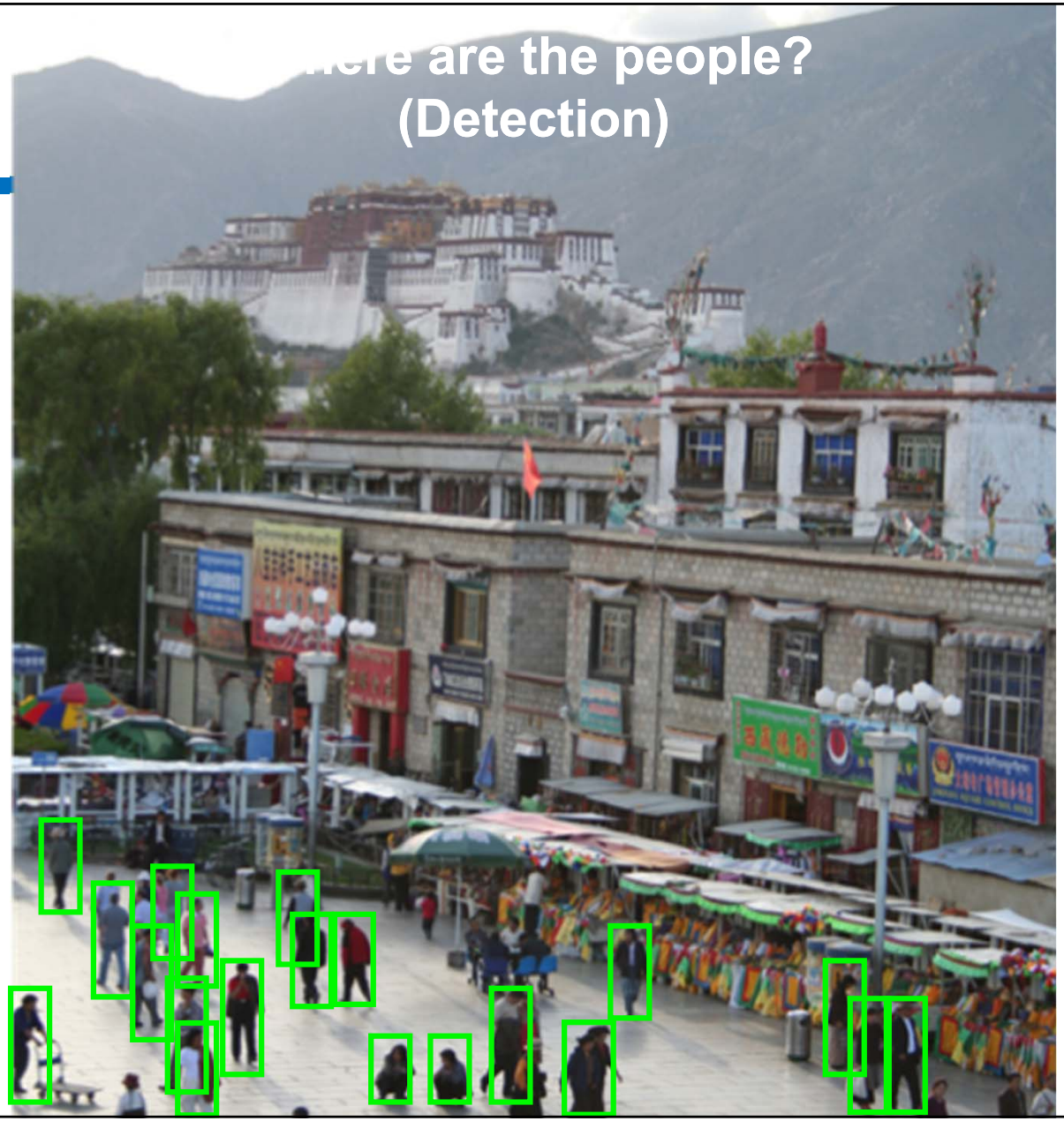


- Bag of words
- Classifier
  - K-Nearest Neighbor
  - Naïve Bayes
  - Support Vector Machine

this a street light?
(Recognition / classification)

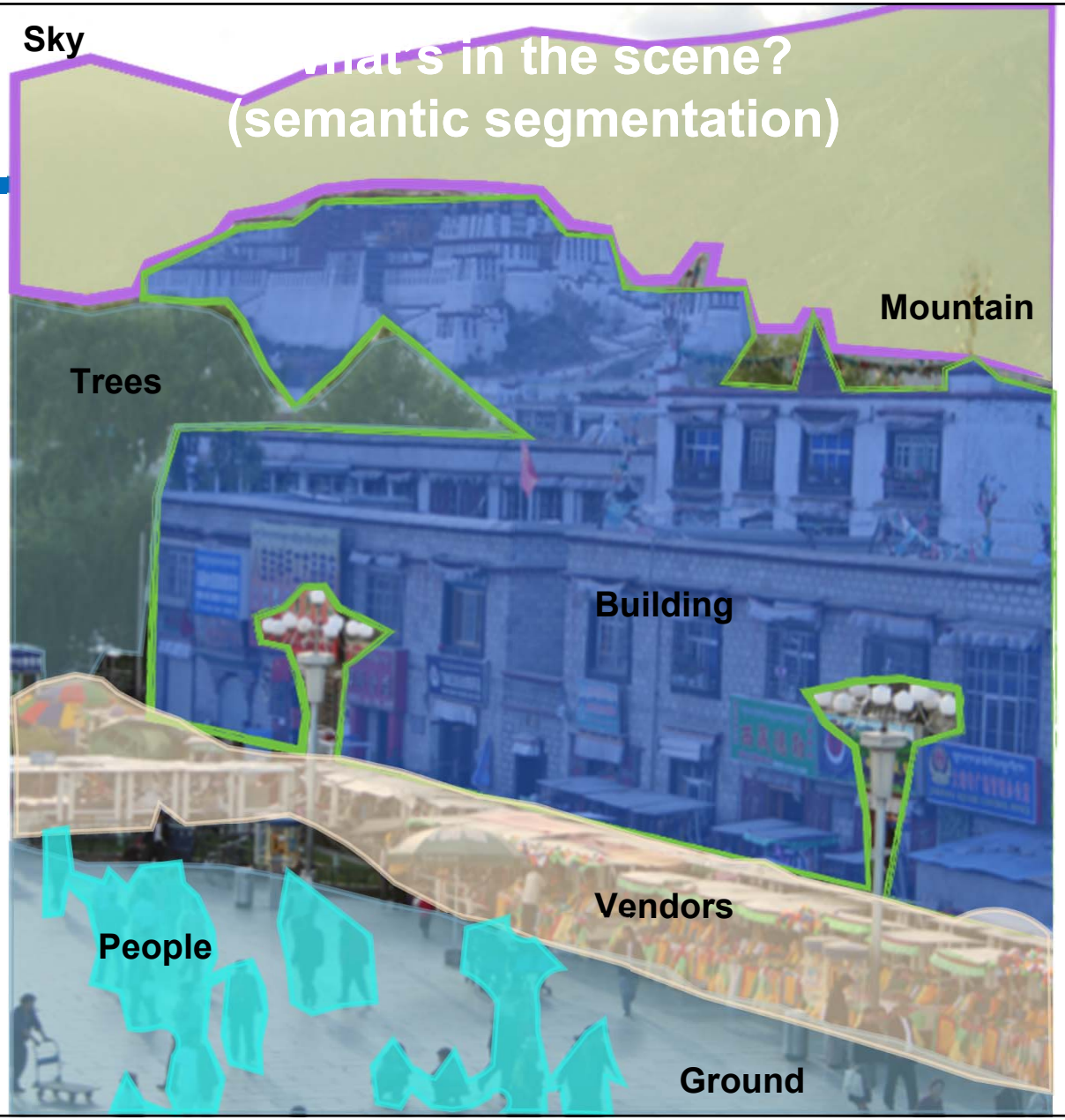


...ere are the people?
(Detection)



hat Potala palace? (Identification)

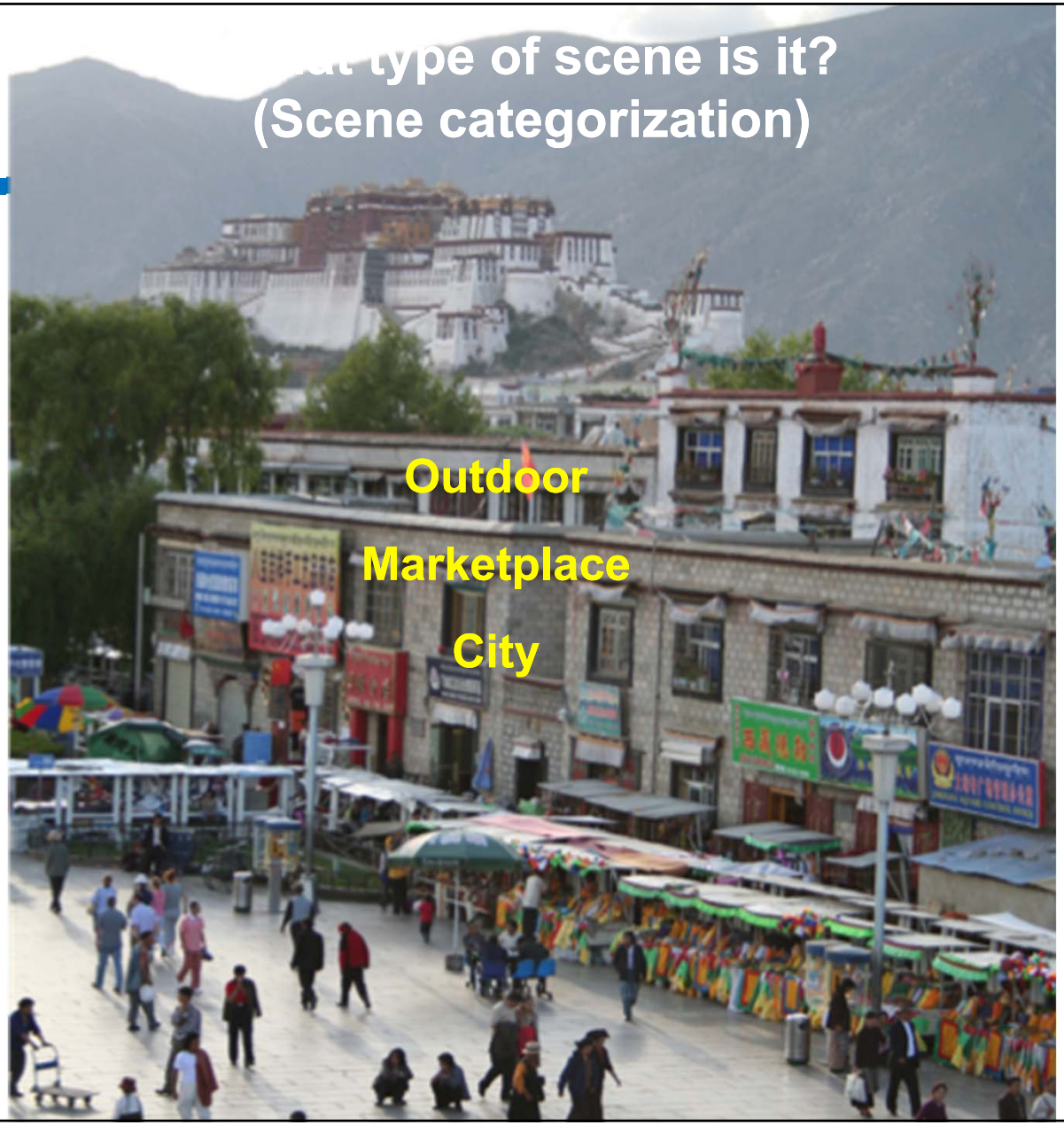



Sky
(semantic segmentation)
Mountain
Trees
Building
Vendors
People
Ground

Object categorization



...at type of scene is it?
(Scene categorization)



Outdoor

Marketplace

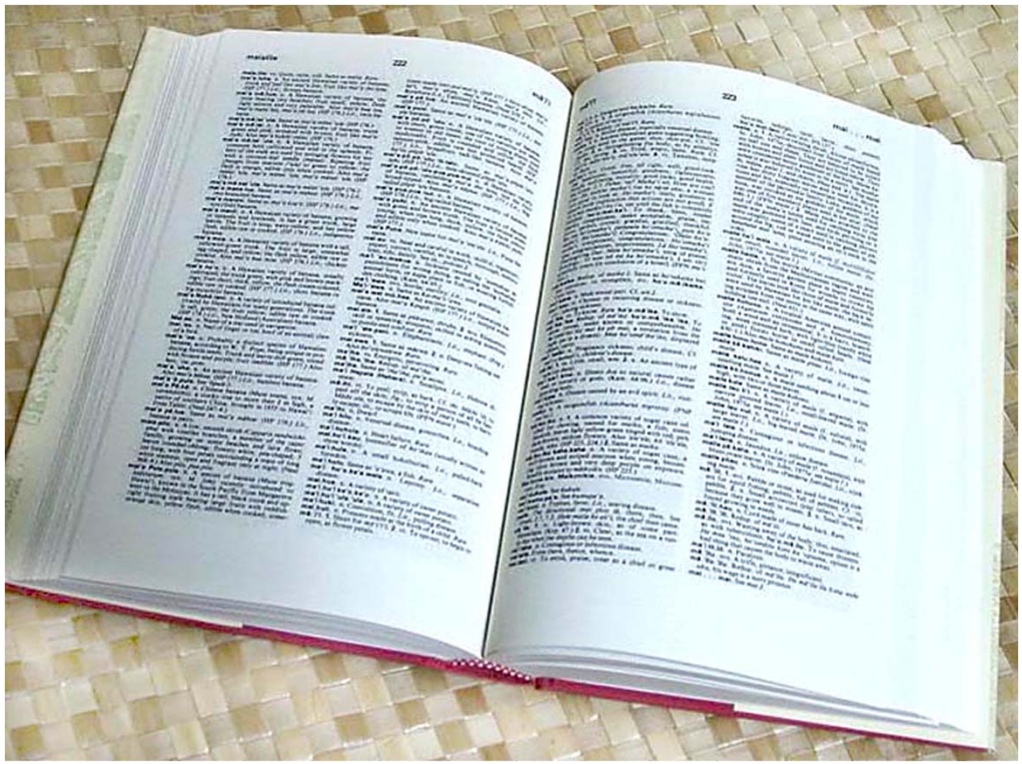
City

Activity / Event Recognition



Object categorization & Recognition

- How many categories?



Object categorization & Recognition

- How many things?



10K+
[Biederman '87]

60K+
product categories

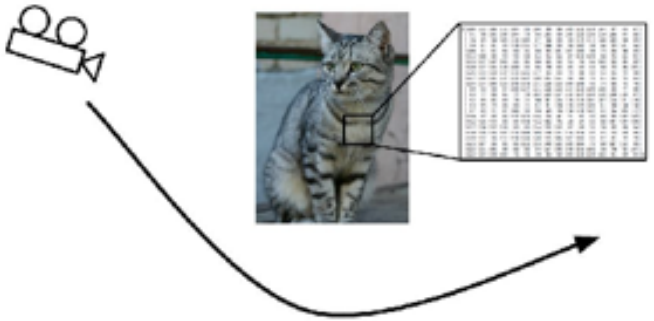
80K+
English nouns
[Miller '95; Fellbaum '98]

3.5M+
unique tags
[Sigurbjörnsson & Zwol '08]

4.1M+
articles

Challenges of recognition

Viewpoint

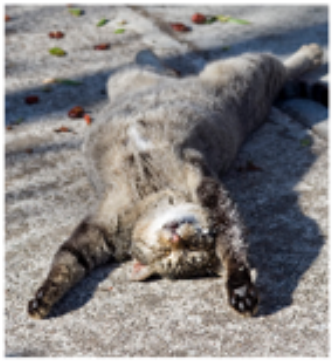


Illumination



This image is CC0 1.0 public domain

Deformation



This image by Umberto Salvagnin is licensed under CC-BY 2.0

Occlusion



This image by jonsson is licensed under CC-BY 2.0

Clutter



This image is CC0 1.0 public domain

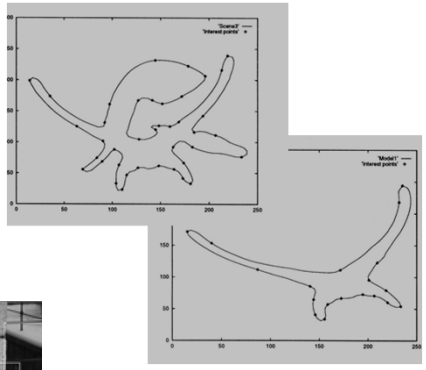
Intraclass Variation

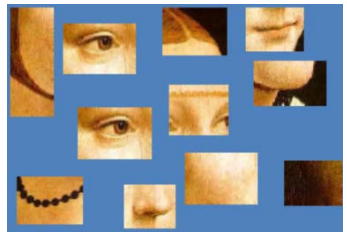


This image is CC0 1.0 public domain

History

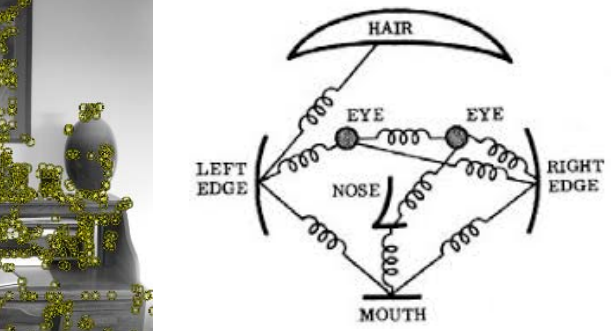
- 1960s – early 1990s: geometry
- 1990s: appearance
- Mid-1990s: sliding window
- Late 1990s: local features
- Early 2000s: parts-and-shape models
- Mid-2000s: bags of features
- Present trends: data-driven methods, context













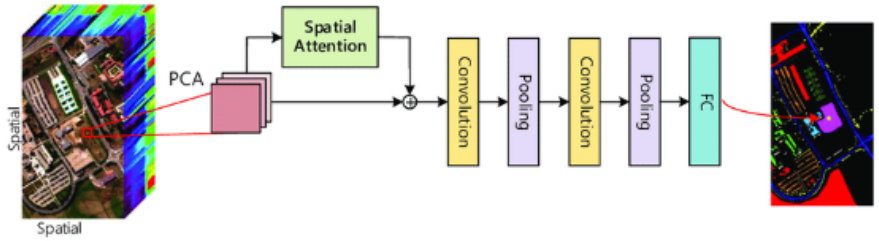


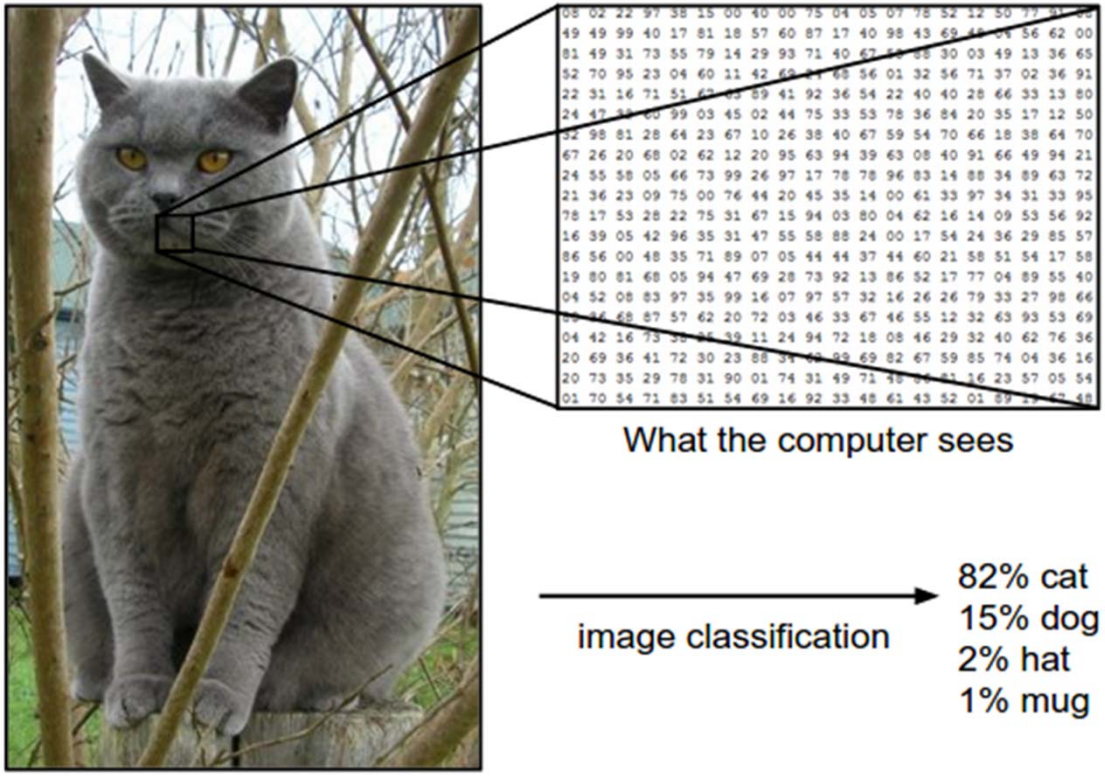
Image Classification



(assume given set of discrete labels)
{dog, cat, truck, plane, ...}

→ cat

Image Classification: Problem



Data-driven approach

- Collect a database of images with labels
- Use ML to train an image classifier
- Evaluate the classifier on test images



Bag of Words

Bag-of-words (BoW models)

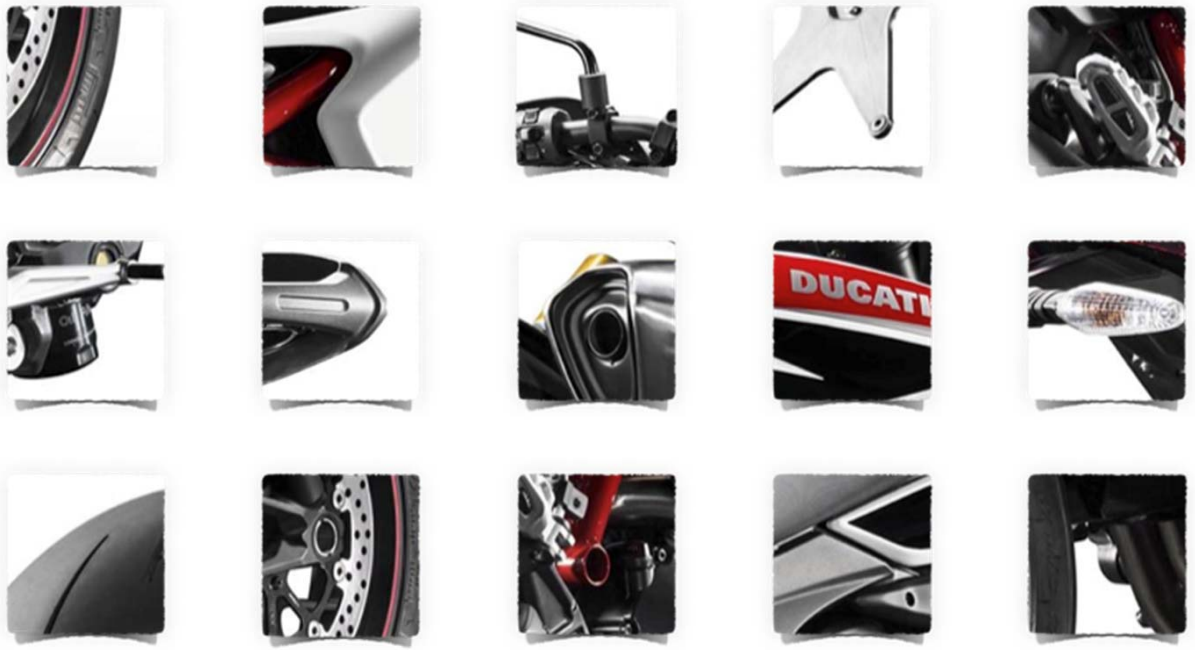
Object → **Bag of 'words'**





Bag-of-words

- What object do these parts belong to?

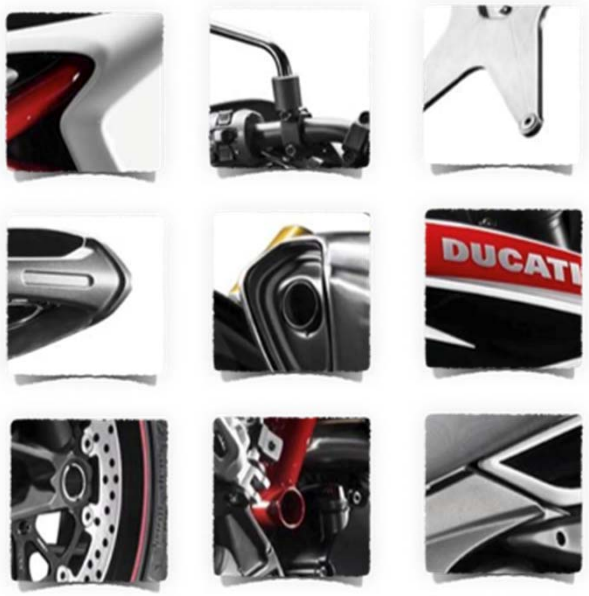


Bag-of-words

Some local feature are very informative

An object as





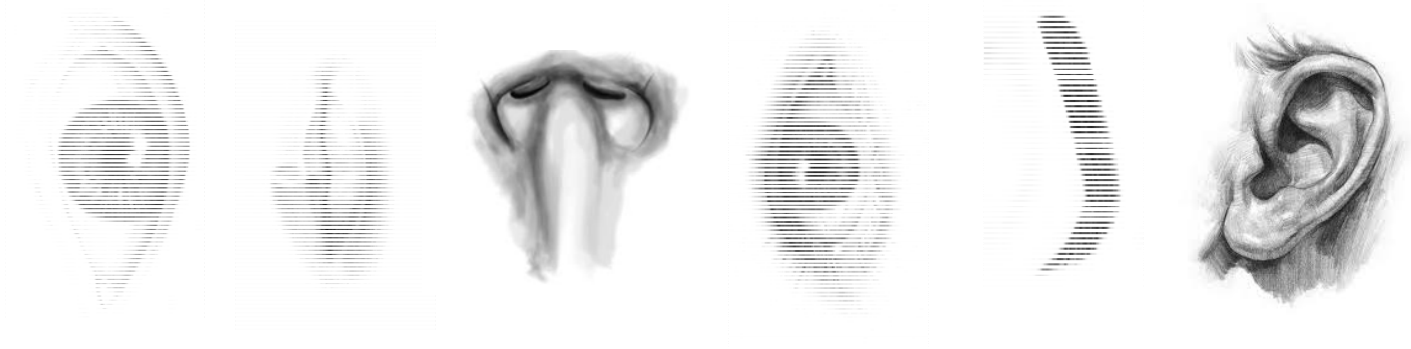
a collection of local features

(bag-of-features)

- deals well with occlusion
- scale invariant
- rotation invariant

Bag-of-words

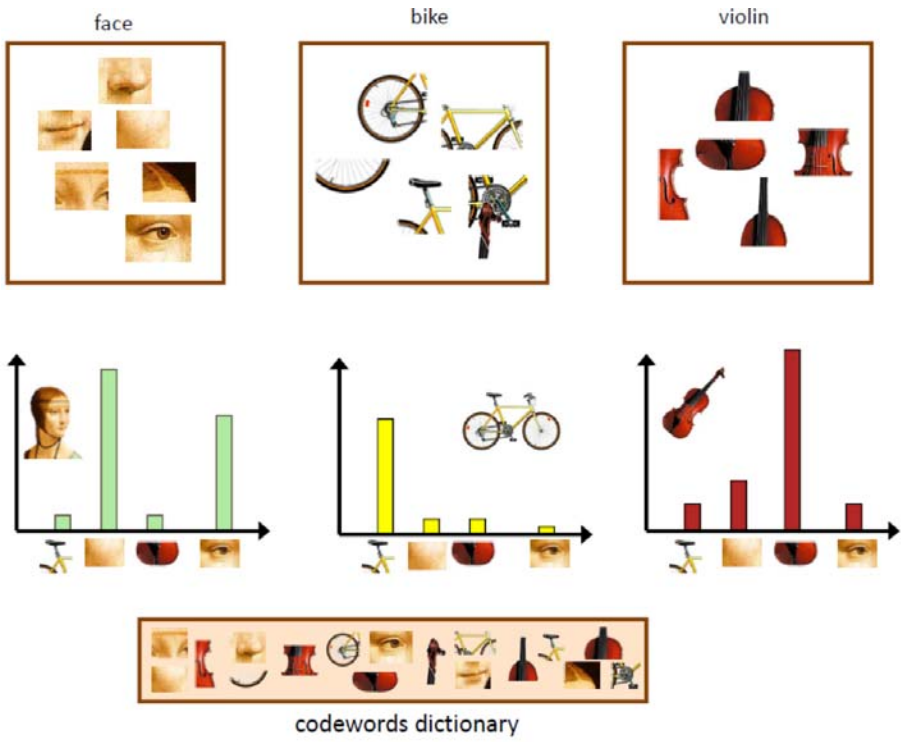
- (not so) crazy assumption



spatial information of local features
can be ignored for object recognition (i.e., verification)

Bag-of-words

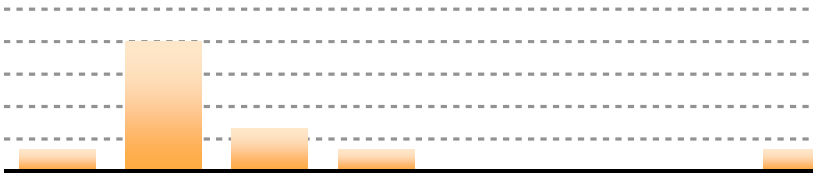
- Definition of BoW
  - Represent a data item (document, texture, image) as a histogram over features
  - Independent features
  - Histogram representation



Bag-of-words

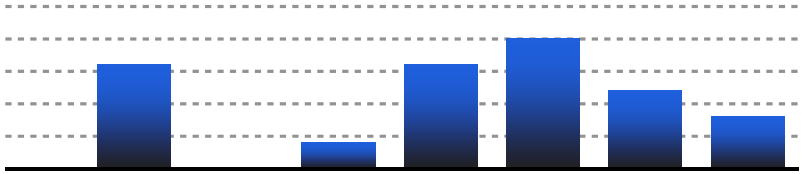
- Vector Space Model





1	6	2	1	0	0	0	1
Tartan	robot	CHIMP	CMU	bio	soft	ankle	sensor





0	4	0	1	4	5	3	2
Tartan	robot	CHIMP	CMU	bio	soft	ankle	sensor

G. Salton. 'Mathematics and Information Retrieval' Journal of Documentation,1979

Bag-of-words

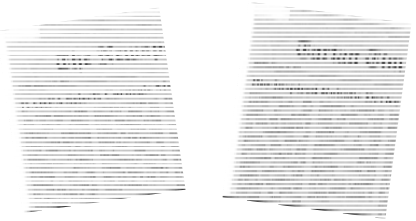
- Vector Space Model
  - A document (datapoint) is a vector of counts over each word (feature)

$$\boldsymbol{v}_d = [n(w_{1,d}) \quad n(w_{2,d}) \quad \cdots \quad n(w_{T,d})]$$

just a histogram over words

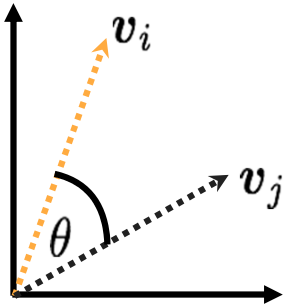
$n(\cdot)$ counts the number of occurrences

  - What is the similarity between two documents?



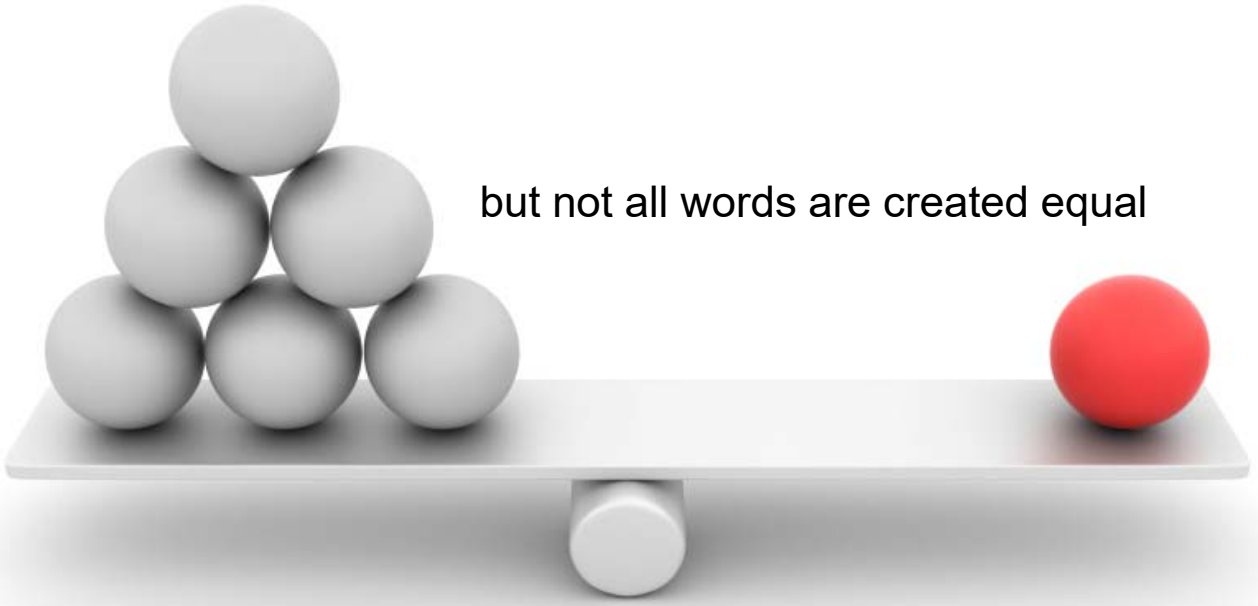
Use any distance you want but the cosine distance is fast.

$$d(\boldsymbol{v}_i, \boldsymbol{v}_j) = \cos\theta = \frac{\boldsymbol{v}_i \cdot \boldsymbol{v}_j}{\|\boldsymbol{v}_i\| \|\boldsymbol{v}_j\|}$$



Bag-of-words

- Importance of Words



Bag-of-words

- TF-IDF
  - **T**erm **F**requency **I**nverse **D**ocument **F**requency

$$\boldsymbol{v}_d = [n(w_{1,d}) \quad n(w_{2,d}) \quad \cdots \quad n(w_{T,d})]$$

weigh each word by a heuristic

$$\boldsymbol{v}_d = [n(w_{1,d})\alpha_1 \quad n(w_{2,d})\alpha_2 \quad \cdots \quad n(w_{T,d})\alpha_T]$$

term frequency

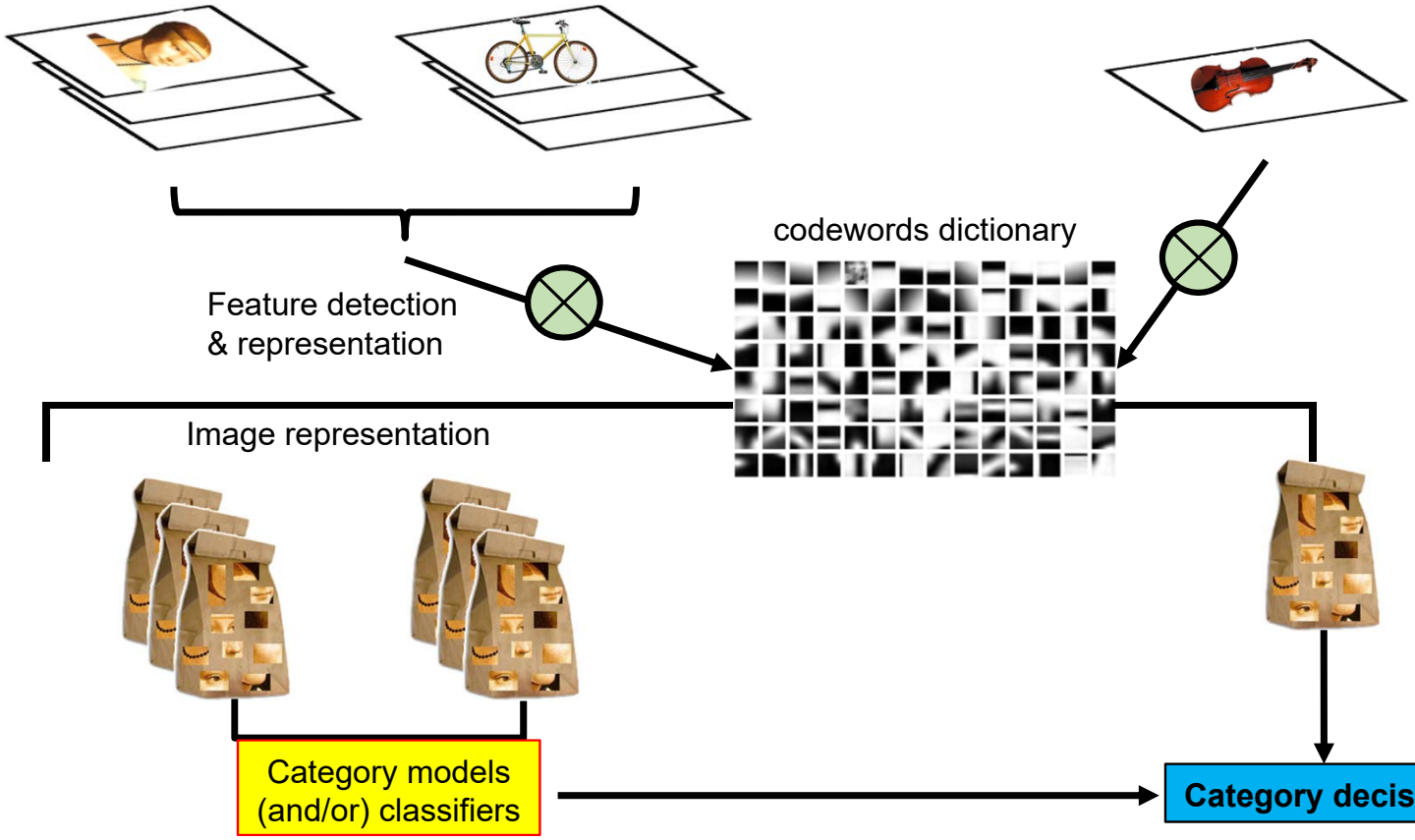
inverse document frequency

$$n(w_{i,d})\alpha_i = n(w_{i,d}) \log \left\{ \frac{D}{\sum_{d'} \mathbf{1}[w_i \in d']} \right\}$$

(down-weights **common** terms)

Bag-of-words

- BOW Pipeline for Image Classification



Bag-of-words

- Standard BOW Pipeline (for Image Classification)

1. **Dictionary Learning:**
   - Learn *Visual Words* using clustering
2. **Encode:**
   - build Bags-of-Words (BOW) vectors for each image
3. **Classify:**
   - Train and test data using BOWs

Bag-of-words

1. **Dictionary Learning:**
   - Learn Visual Words using clustering

1. extract features (e.g., SIFT) from images



2. Learn visual dictionary (e.g., K-means clustering)

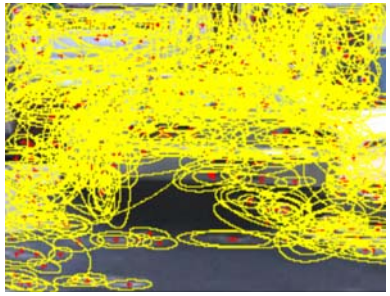


Bag-of-words

- Feature detection and representation
  - Regular grid



  - Interest point detector



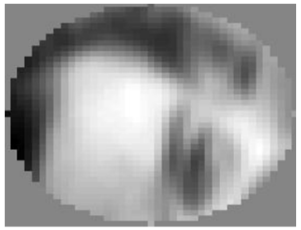
  - Other methods: Random sampling & segmentation based patches

Bag-of-words

- Feature detection and representation



Detect patches



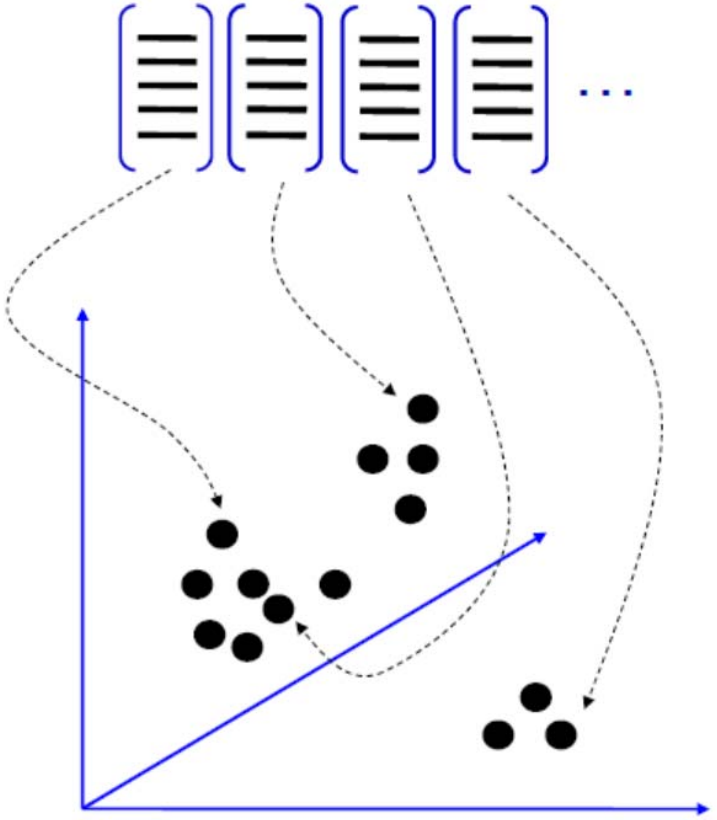
Normalize patch

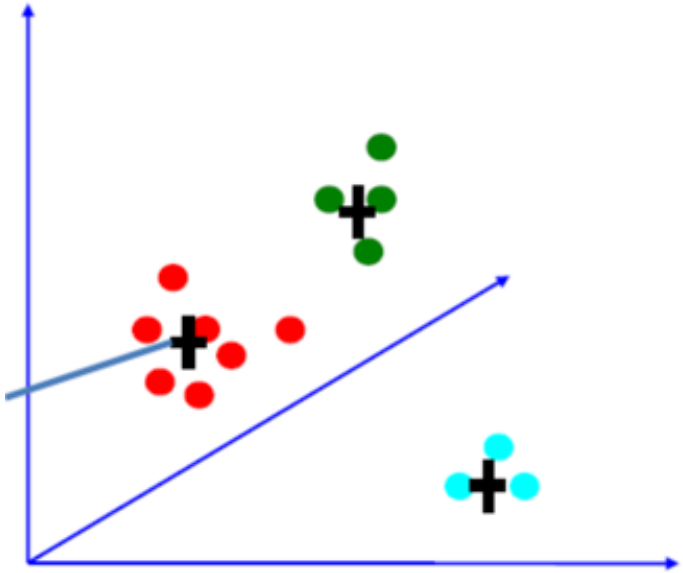


Compute descriptor

Bag-of-words

- Codewords dictionary formation

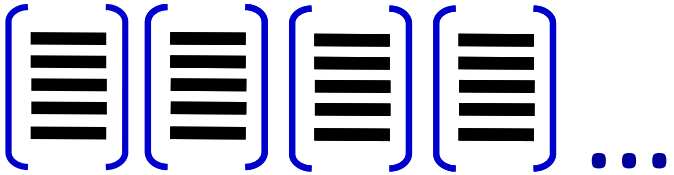


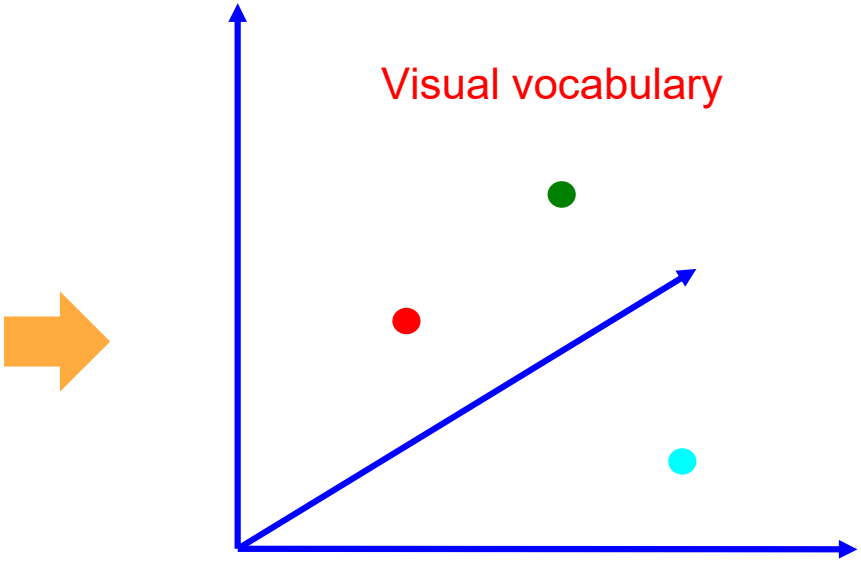


Clustering /
Vector Quantization

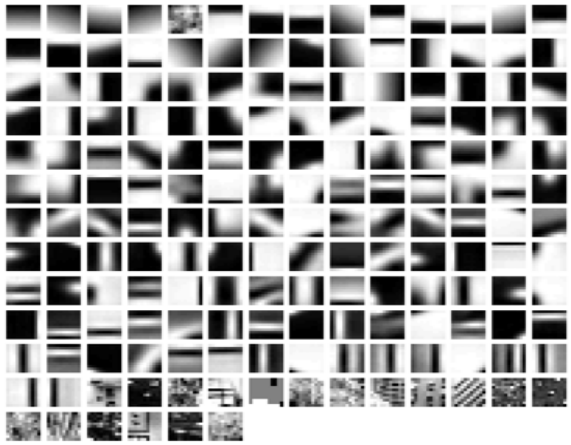
Bag-of-words

- Codewords dictionary formation





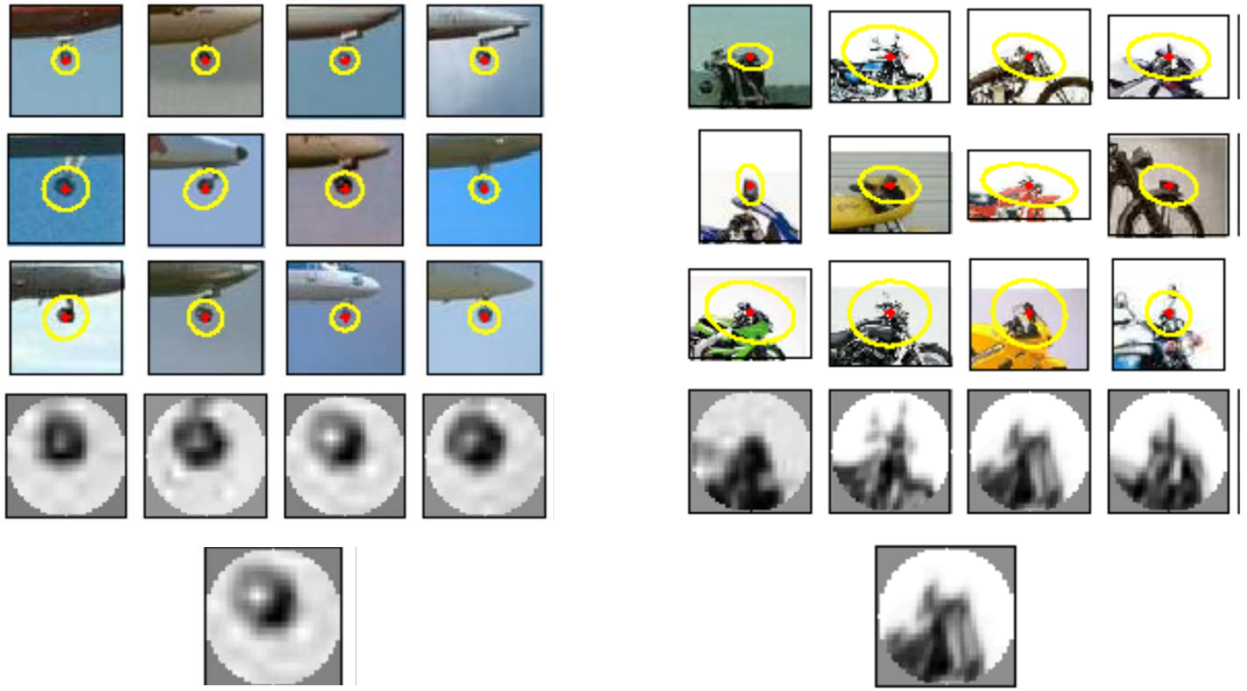
- Visual Dictionary
  - From what **data** should I learn the dictionary?
    - Dictionary can be learned on separate training set
    - Provided the training set is sufficiently representative, the dictionary will be “universal”



Example of Visual Dictionary

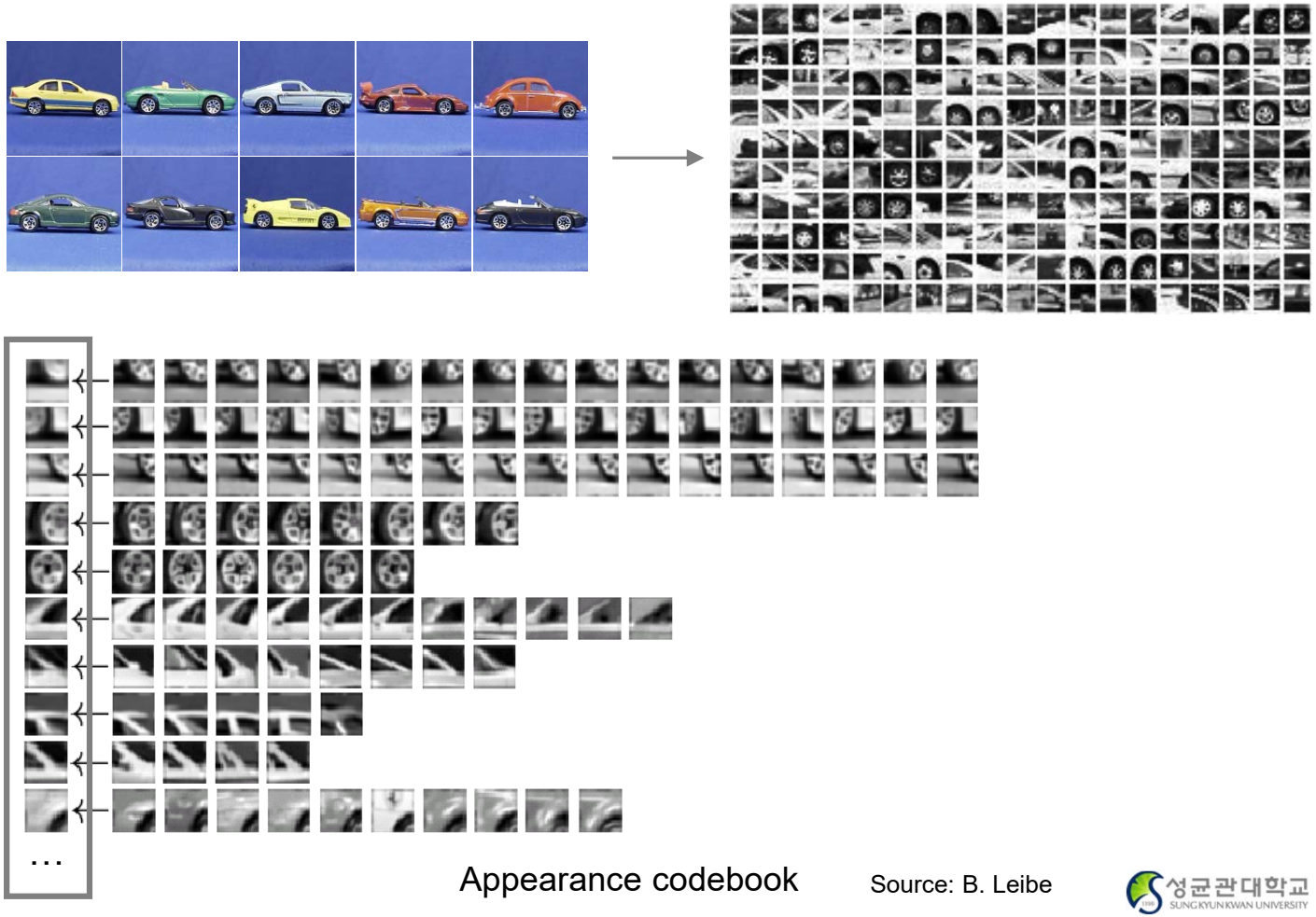
Codewords dictionary formation

- Codewords dictionary formation
  - Image patch examples of codewords



Bag-of-words

- Visual Dictionary
  - example



Appearance codebook

Source: B. Leibe

Bag-of-words

- Visual Dictionary
  - example

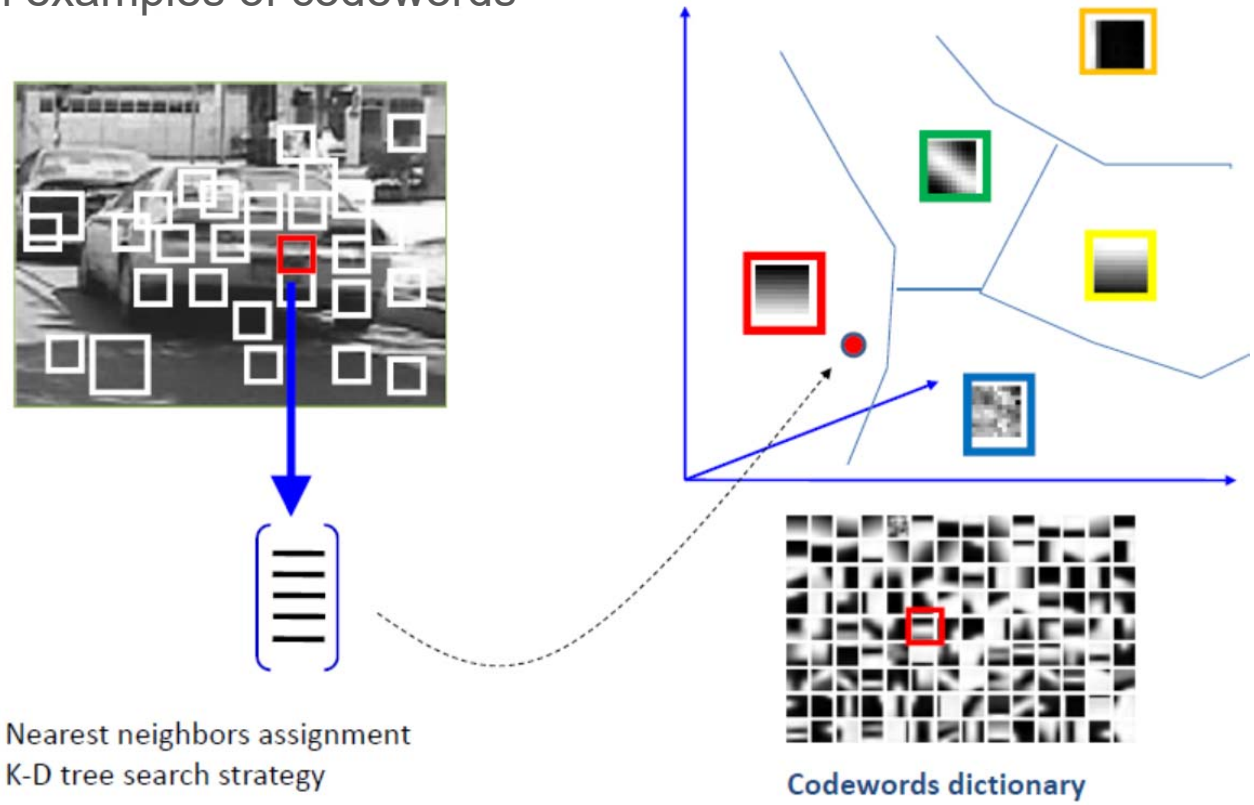


Appearance codebook

Source: B. Leibe

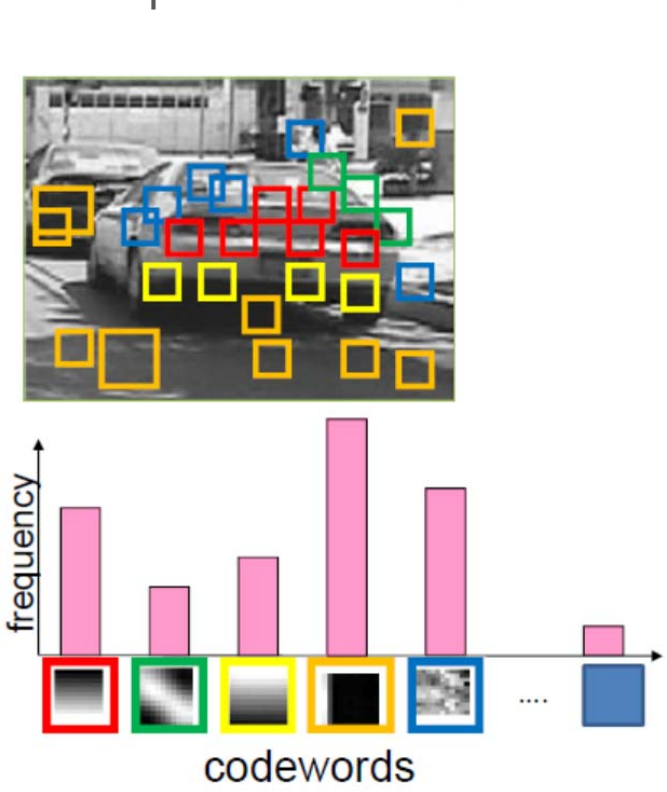
Bag of word representation

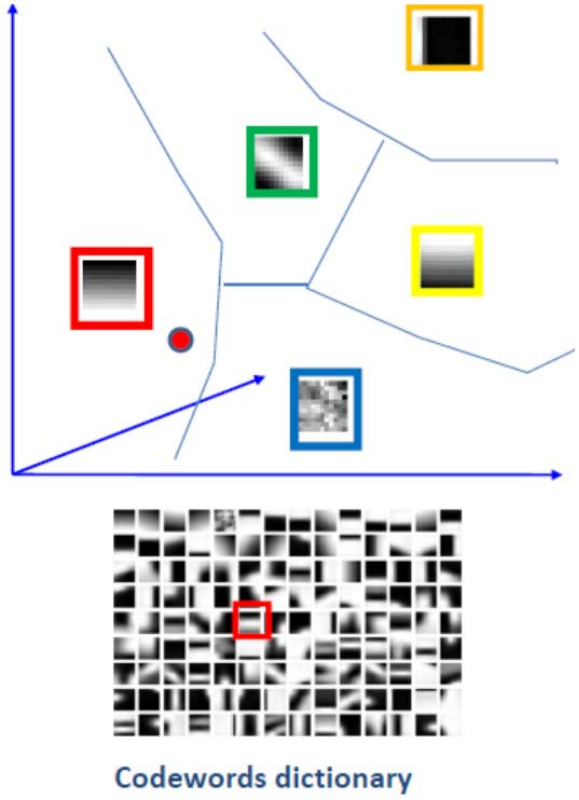
- Bag of word representation
  - Image patch examples of codewords



Bag of word representation

- Bag of word representation
  - Image patch examples of codewords

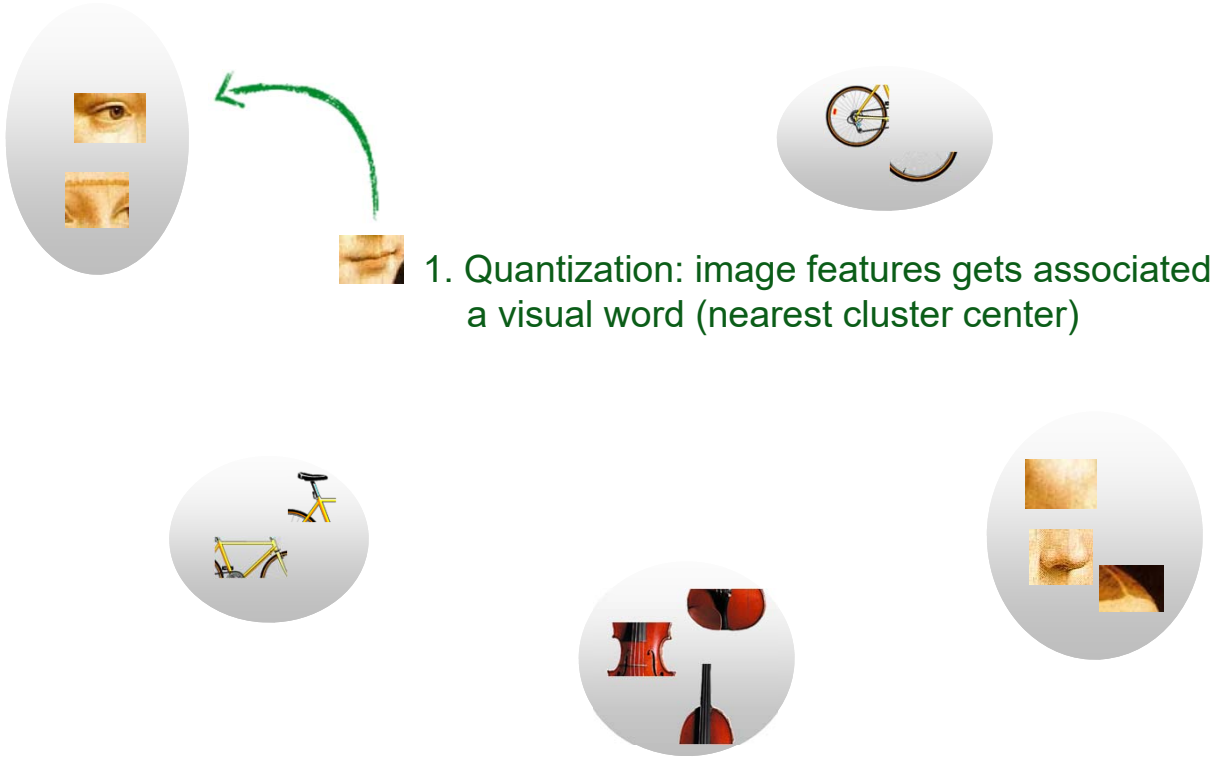




Bag-of-words

2. **Encode:**

- build Bags-of-Words (BOW) vectors for each image



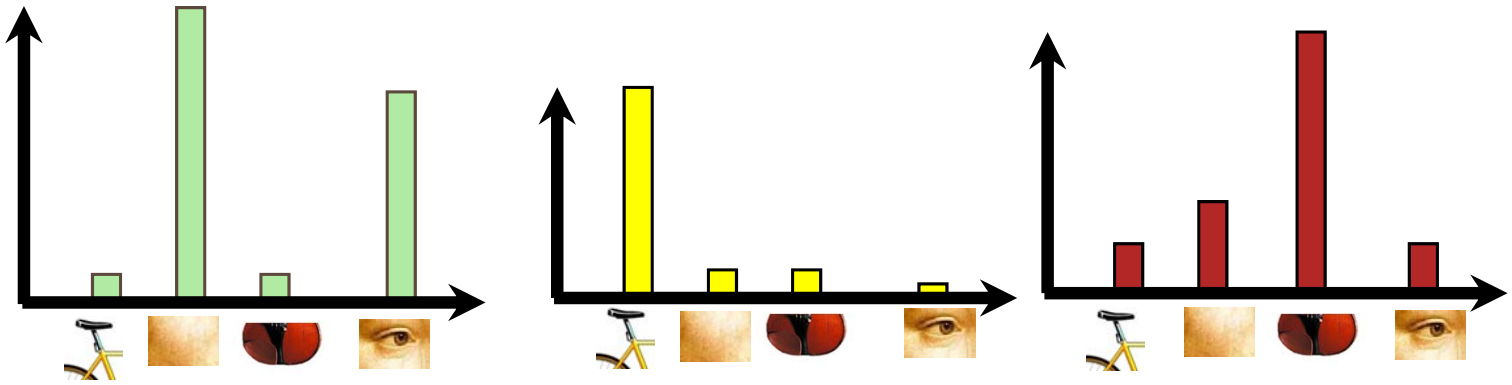
1. Quantization: image features gets associated to a visual word (nearest cluster center)

Bag-of-words

2. **Encode:**

- build Bags-of-Words (BOW) vectors for each image

2. Histogram: count the number of visual word occurrences



Bag-of-words

3. **Classify:**

- Train and test data using BOWs

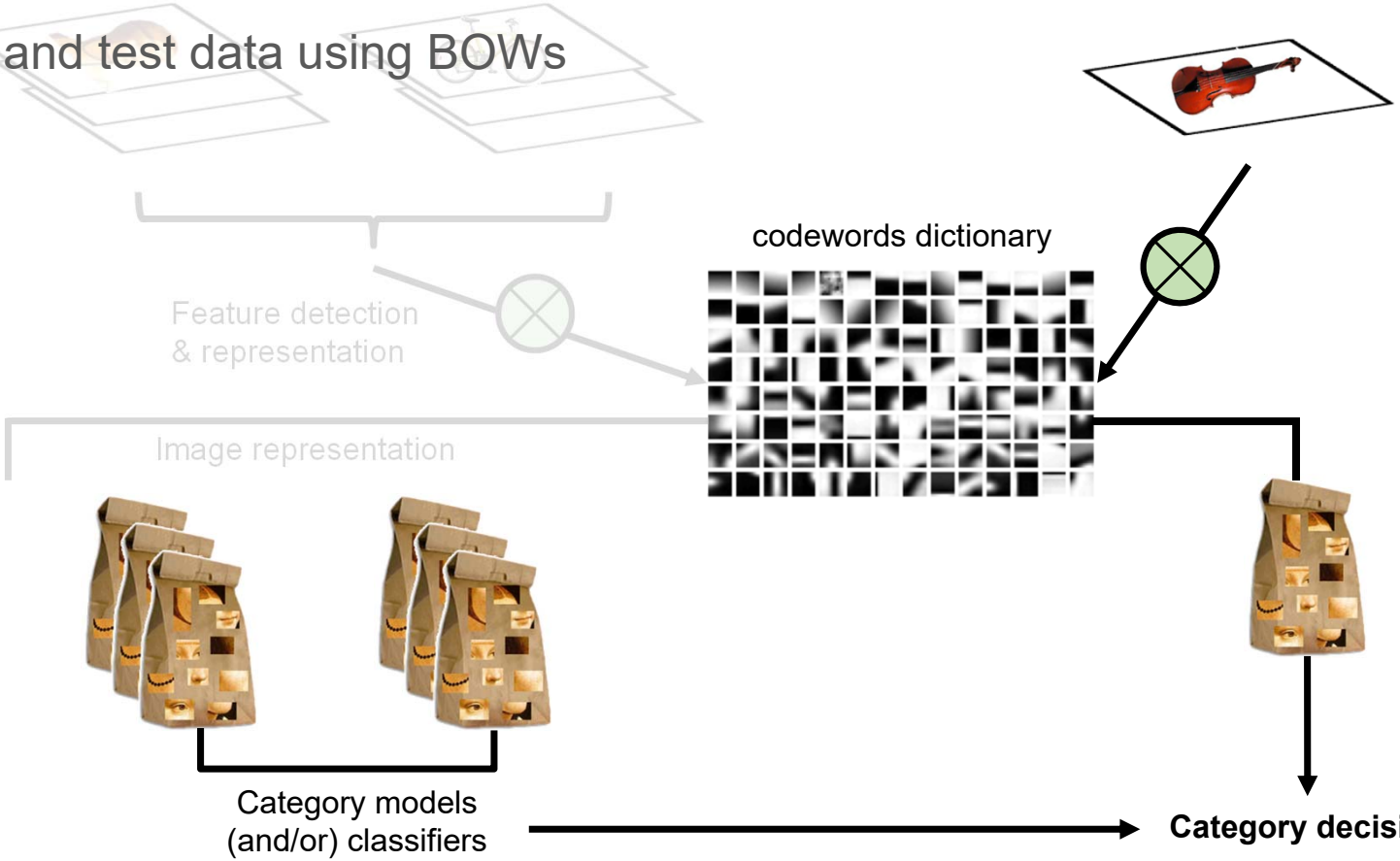
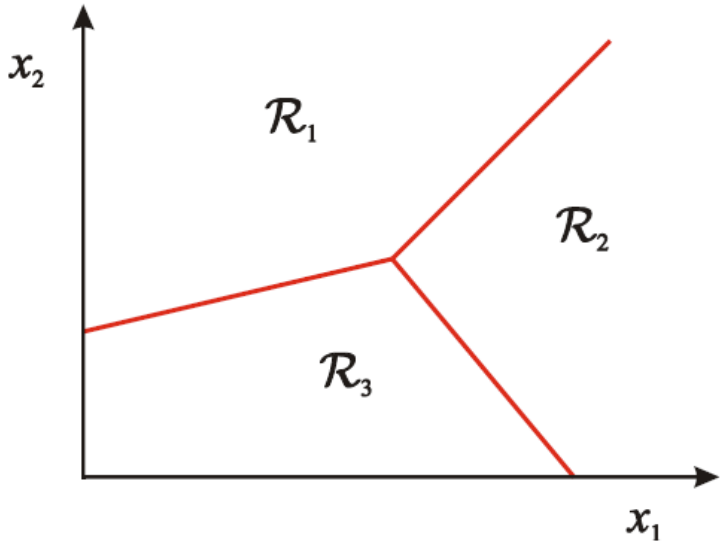


Image Classification

Classification

- Assign input vector to one of two or more classes
- Any decision rule divides input space into *decision regions* separated by *decision boundaries*

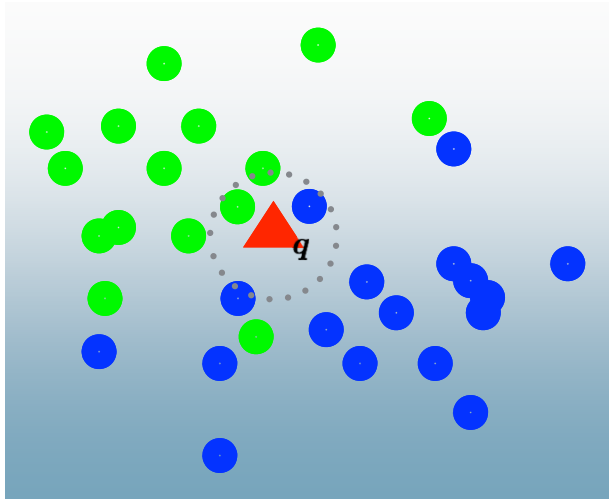


- Classifiers
  - Nearest Neighbor
  - kNN (k-Nearest Neighbors)
  - Support Vector Machine
  - Neural Network
  - Deep Neural Network

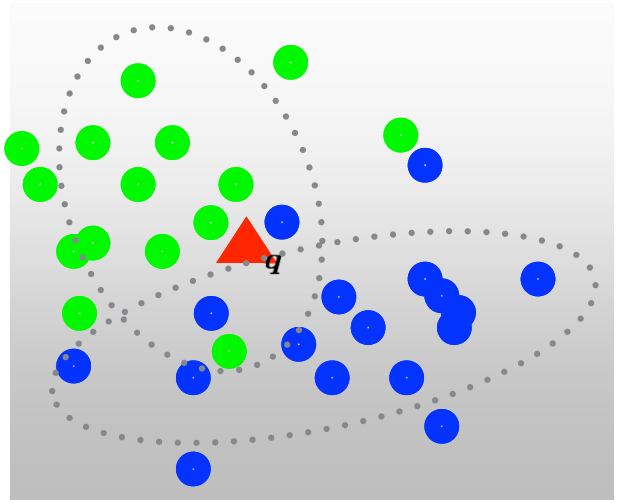
  .....

Classification

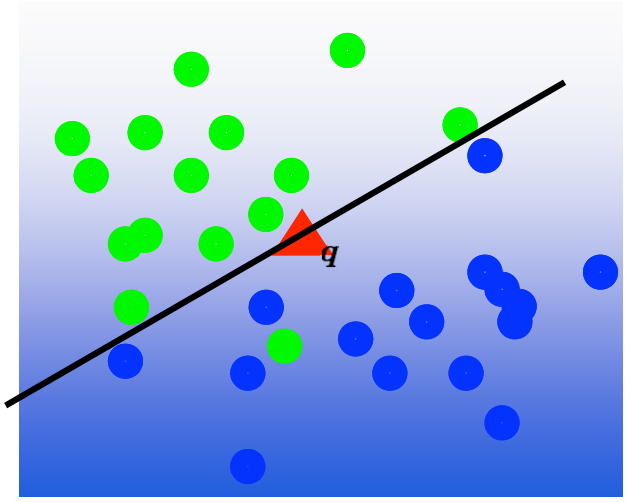
K-Nearest Neighbors



Naïve Bayes

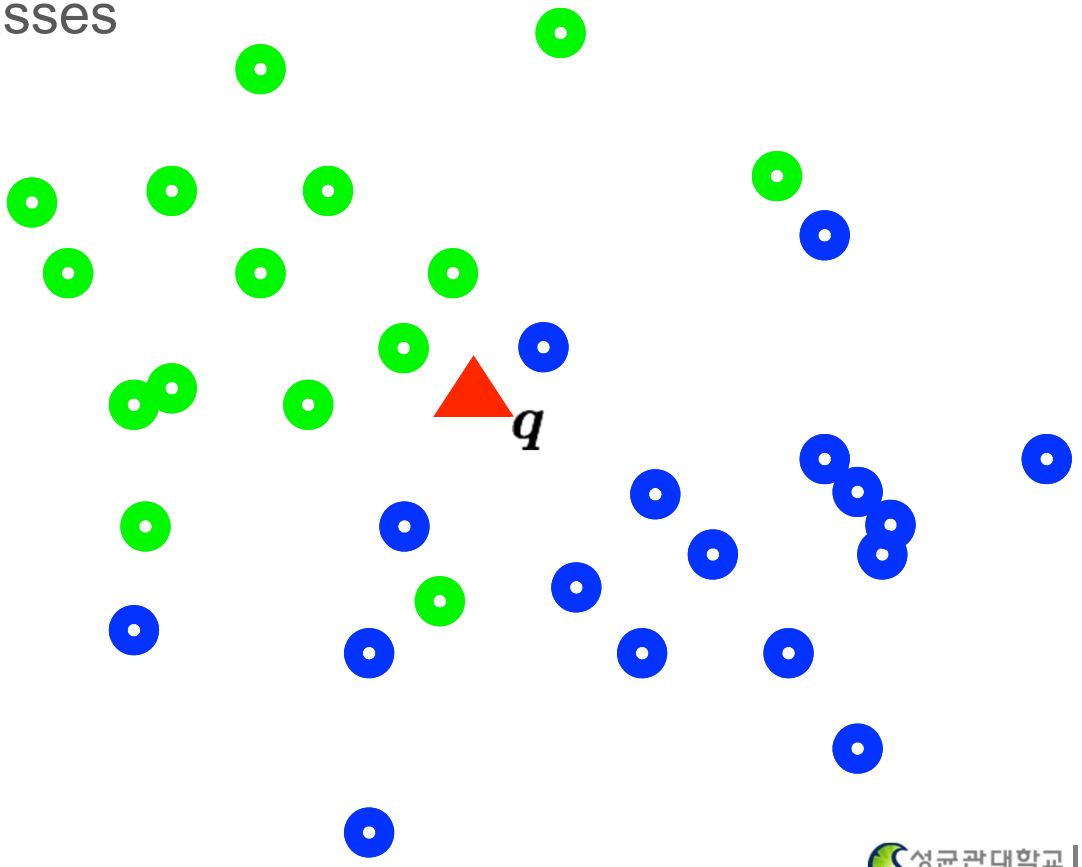


Support Vector Machine



K-Nearest Neighbors

- Distribution of data from two classes

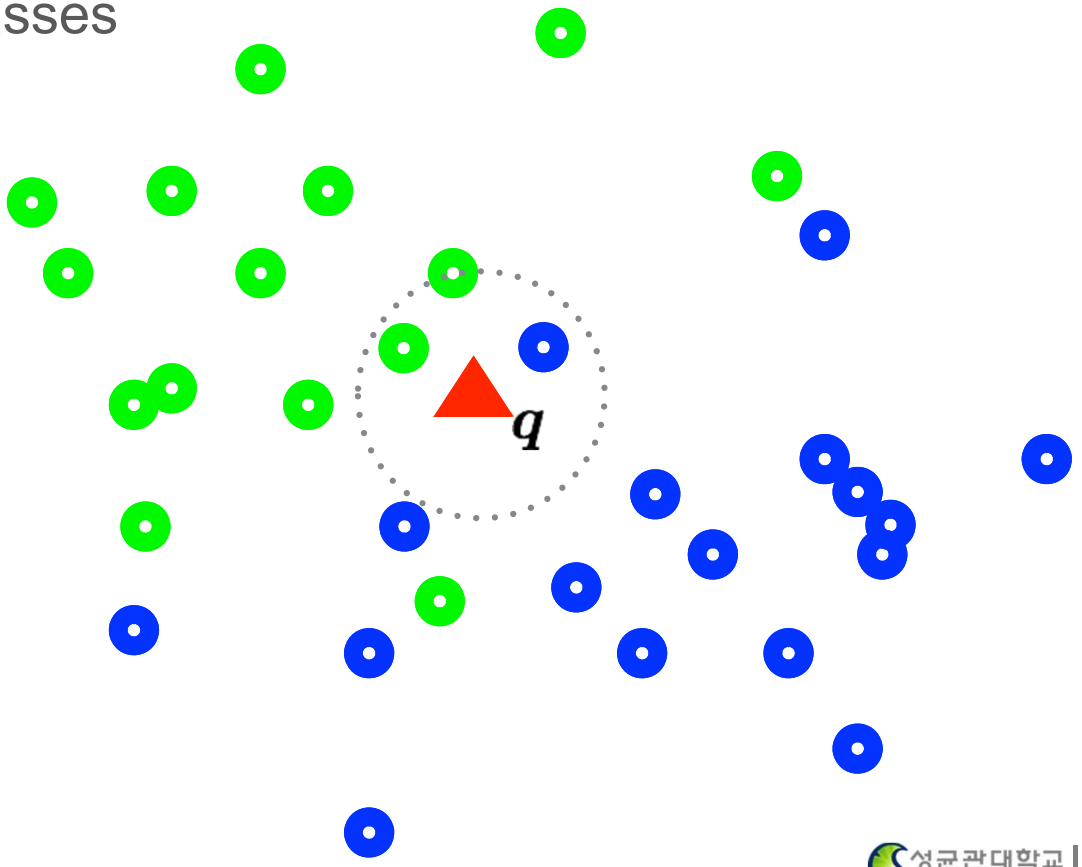


Which class does q belong too?

K-Nearest Neighbors

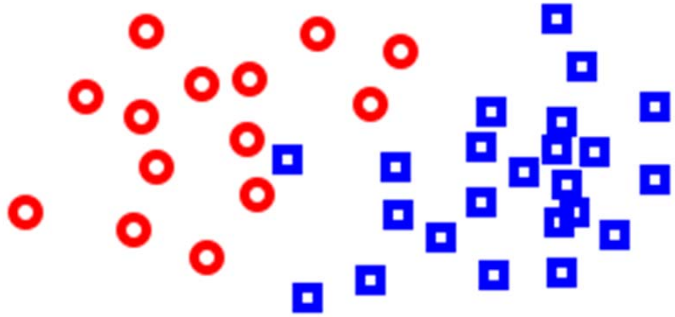
- Distribution of data from two classes

Look at the neighbors



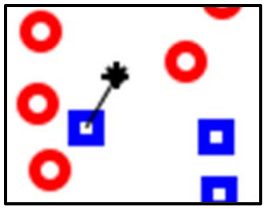
K-Nearest Neighbors

- Non-parametric pattern classification approach
- Consider a two class problem where each sample consists of two measurements (x, y)



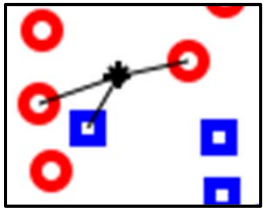
- For a given query point q, assign the class of the nearest neighbor

k = 1



- Compute the k nearest neighbors and assign the class by majority vote

k = 3



K-Nearest Neighbors

- What is the best distance metric between data points?
  - Typically Euclidean distance
  - Locality sensitive distance metrics
  - Important to normalize. Dimensions have different scales

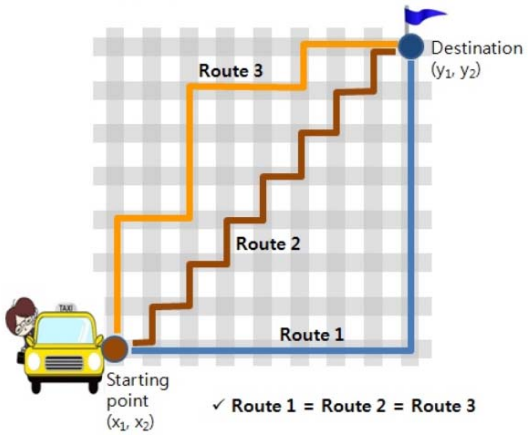
- How many ?
  - Typically K =1 is good
  - Cross-validation (try different K)

K-Nearest Neighbors

- Choice of distance metric

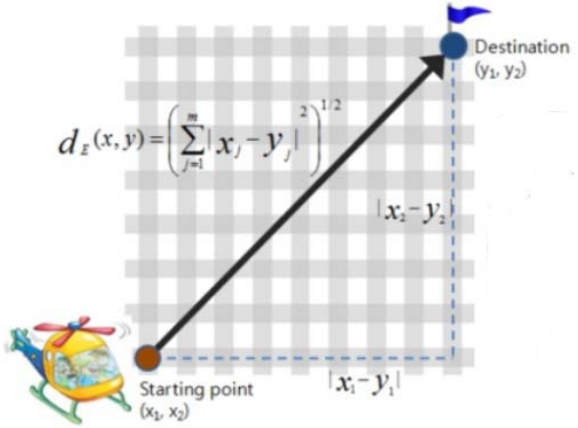
L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



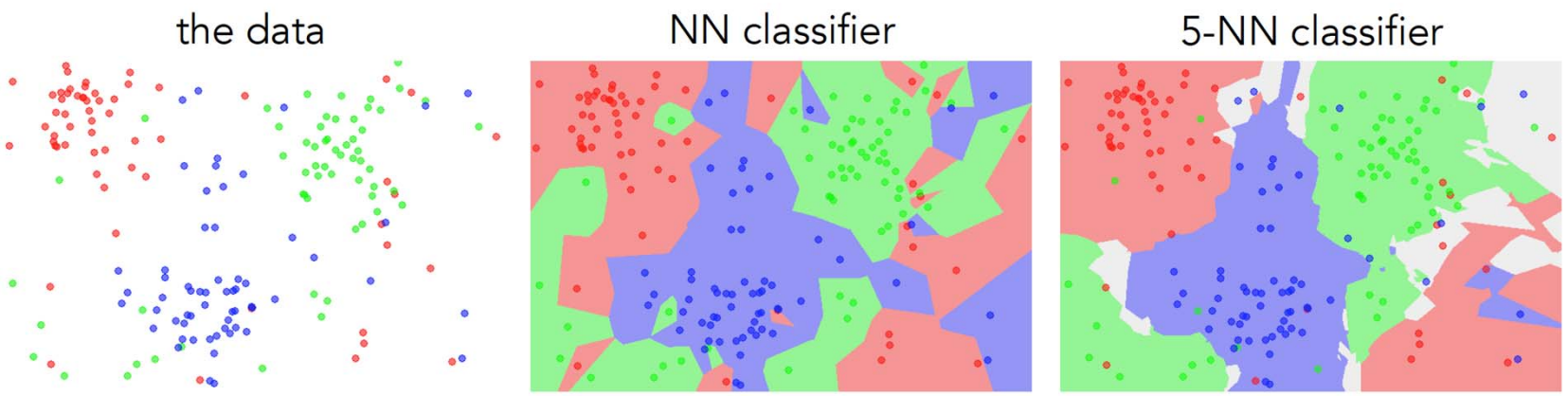
L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$



K-Nearest Neighbors

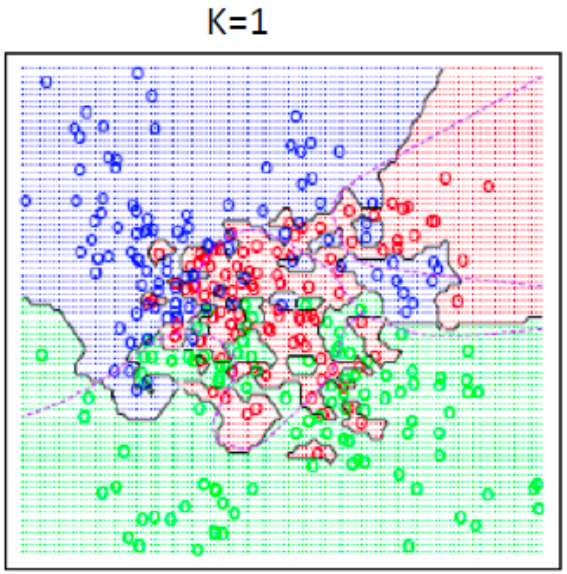
- Example

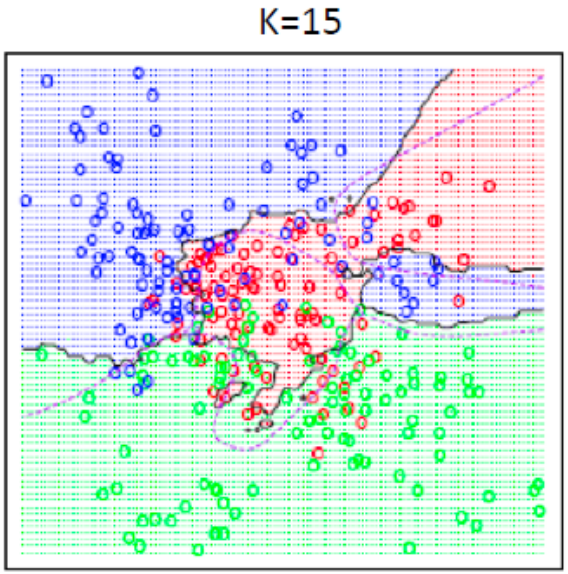


K-Nearest Neighbors

https://opencv24-python-tutorials.readthedocs.io/en/stable/py_tutorials/py_ml/py_knn/py_knn_index.html

- Issues to keep in mind : Choosing the value of k
  - If too small, sensitive to noise points
  - If too large, neighborhood may include points from other classes

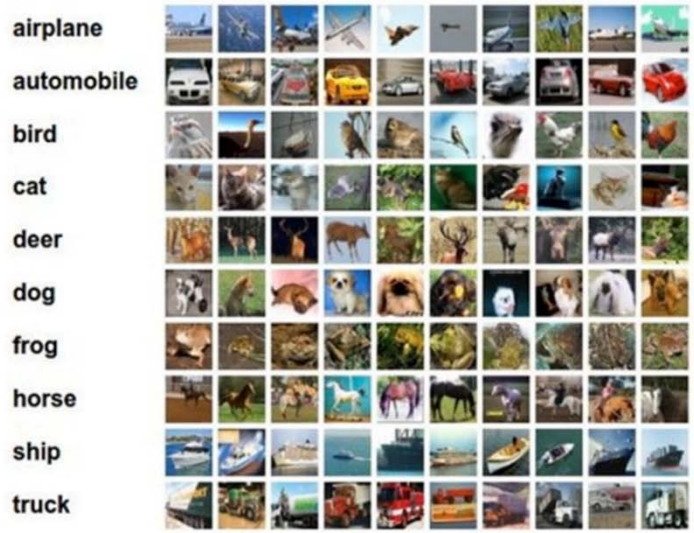




K-Nearest Neighbors

- CIFAR-10 and NN results

Example dataset: **CIFAR-10**
10 labels
50,000 training images
10,000 test images.

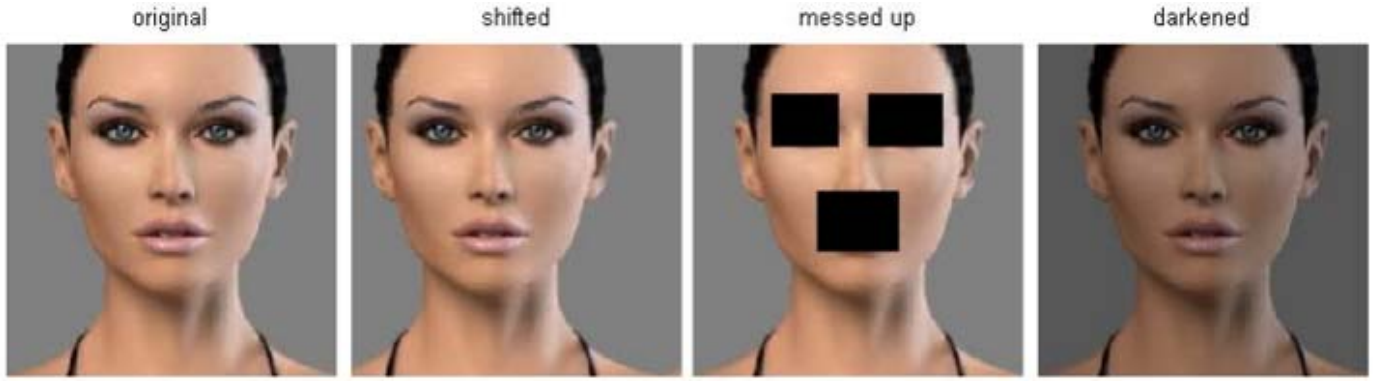


For every test image (first column), examples of nearest neighbors in rows



K-Nearest Neighbors

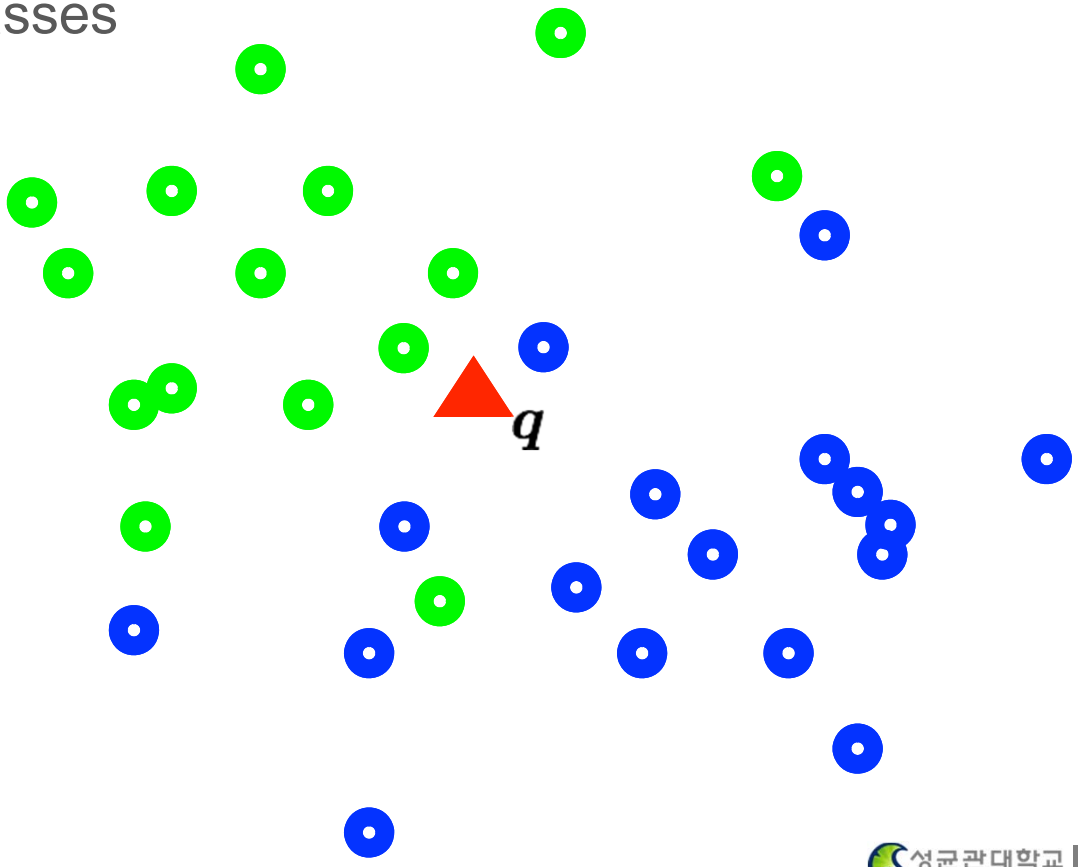
- KNN on images never used
  - Terrible performance at test time
  - Distance metrics on level of whole images can be very unintuitive



(all 3 images have same L2 distance to the one on the left)

Naïve Bayes

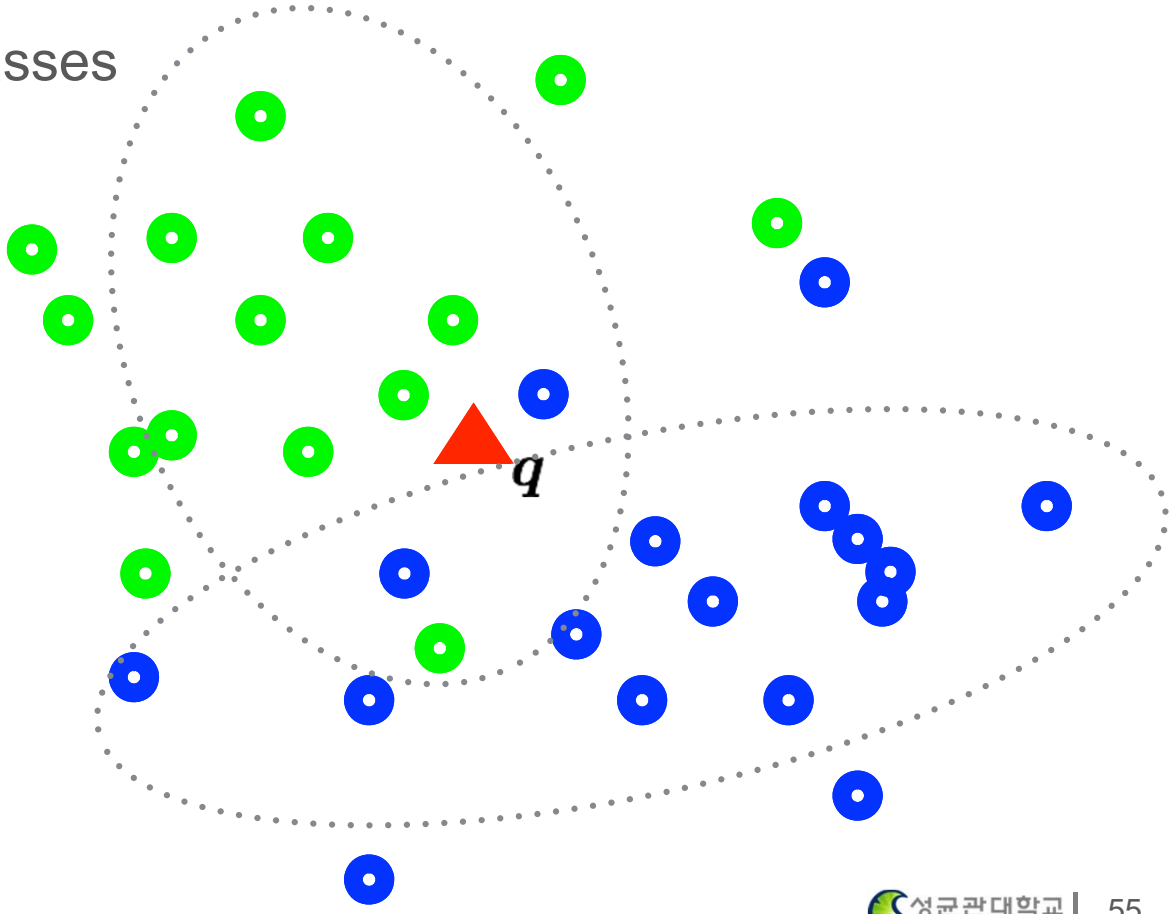
- Distribution of data from two classes



Which class does q belong too?

Naïve Bayes

- Distribution of data from two classes



Learn parametric model for each class
Compute probability of query

Naïve Bayes

- This is called the posterior:
- the probability of a class z given the observed features X

$$p(z|X)$$

For classification, z is a discrete random variable
(e.g., car, person, building)

X is a set of observed features
(e.g., features from a single image)

(it's a function that returns a single probability value)

Naïve Bayes

- The posterior can be decomposed according to **Bayes' Rule**

likelihood prior

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)}$$

posterior

Marginal probability

In our context…

$$p(\boldsymbol{z}|\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N) = \frac{p(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N|\boldsymbol{z})p(\boldsymbol{z})}{p(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N)}$$

$$\hat{z} = \arg\max_{z \in \boldsymbol{z}} p(\boldsymbol{X}|z)p(z)$$

To optimize this…we need to compute this

Compute the likelihood…

Naïve Bayes

- To compute the MAP estimate

Given (1) a set of known parameters (2) observations

$$p(\boldsymbol{z}) \quad p(\boldsymbol{x}|\boldsymbol{z}) \qquad \{x_1, x_2, \ldots, x_N\}$$

Compute which z has the largest probability

$$\hat{z} = \arg\max_{z \in \boldsymbol{z}} p(\boldsymbol{X}|z)p(z)$$

Assume the conditional independence

$$\hat{z} = \arg\max_{z \in \boldsymbol{z}} p(z) \prod_n p(x_n|z)$$

Naïve Bayes

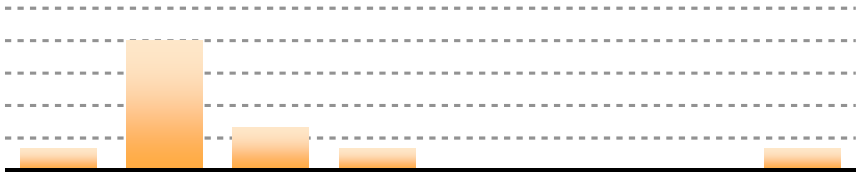


The Newspa

Sunday, December 22, 2013

DARPA Selects Carnegie Me

The Tartan Rescue Team from Carnegie Mellon University's National Robotics Engineering Center ranked third among teams competing in the Defense Advanced Research Projects Agency (DARPA) Robotics Challenge Trials this weekend in Homestead, Fla., and was selected by the agency as one of eight teams eligible for DARPA funding to prepare for next December's finals. The team's four-limbed CMU Highly Intelligent Mobile Platform, or CHIMP, robot scored 18 out of a possible 32 points during the two-day trials. It demonstrated its ability to perform such tasks as removing debris, cutting a hole through a wall and closing a series of valves.



count	1	6	2	1	0	0	0	1
word	Tartan	robot	CHIMP	CMU	bio	soft	ankle	sensor
p(x|z)	0.09	0.55	0.18	0.09	0.0	0.0	0.0	0.09

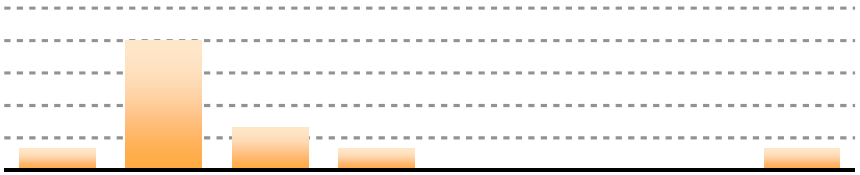
$$p(X|z) = \prod_v p(x_v|z)^{c(w_v)}$$
$$= (0.09)^1(0.55)^6 \cdots (0.09)^1$$

Numbers get really small so use log probabilities

$$\log p(X|z = \text{'grandchallenge'}) = -2.42 - 3.68 - 3.43 - 2.42 - 0.07 - 0.07 - 0.07 - 2.42 = -14.58$$
$$\log p(X|z = \text{'softrobot'}) = -7.63 - 9.37 - 15.18 - 2.97 - 0.02 - 0.01 - 0.02 - 2.27 = -37.48$$

Naïve Bayes

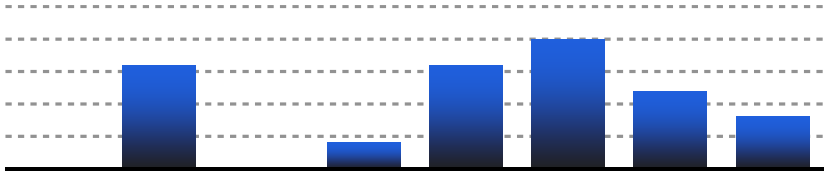




count	1	6	2	1	0	0	0	1
word	Tartan	robot	CHIMP	CMU	bio	soft	ankle	sensor
p(x|z)	0.09	0.55	0.18	0.09	0.0	0.0	0.0	0.09

log p(X|z=grand challenge) = **- 14.58**
log p(X|z=bio inspired) = - 37.48





count	0	4	0	1	4	5	3	2
word	Tartan	robot	CHIMP	CMU	bio	soft	ankle	sensor
p(x|z)	0.0	0.21	0.0	0.05	0.21	0.26	0.16	0.11

log p(X|z=grand challenge) = - 94.06
log p(X|z=bio inspired) = **- 32.41**

Support Vector Machine



class scores

- Linear Classifier

define a **score function**

data (histogram)

$$f(x_i, W, b) = Wx_i + b$$

class scores

"weights"

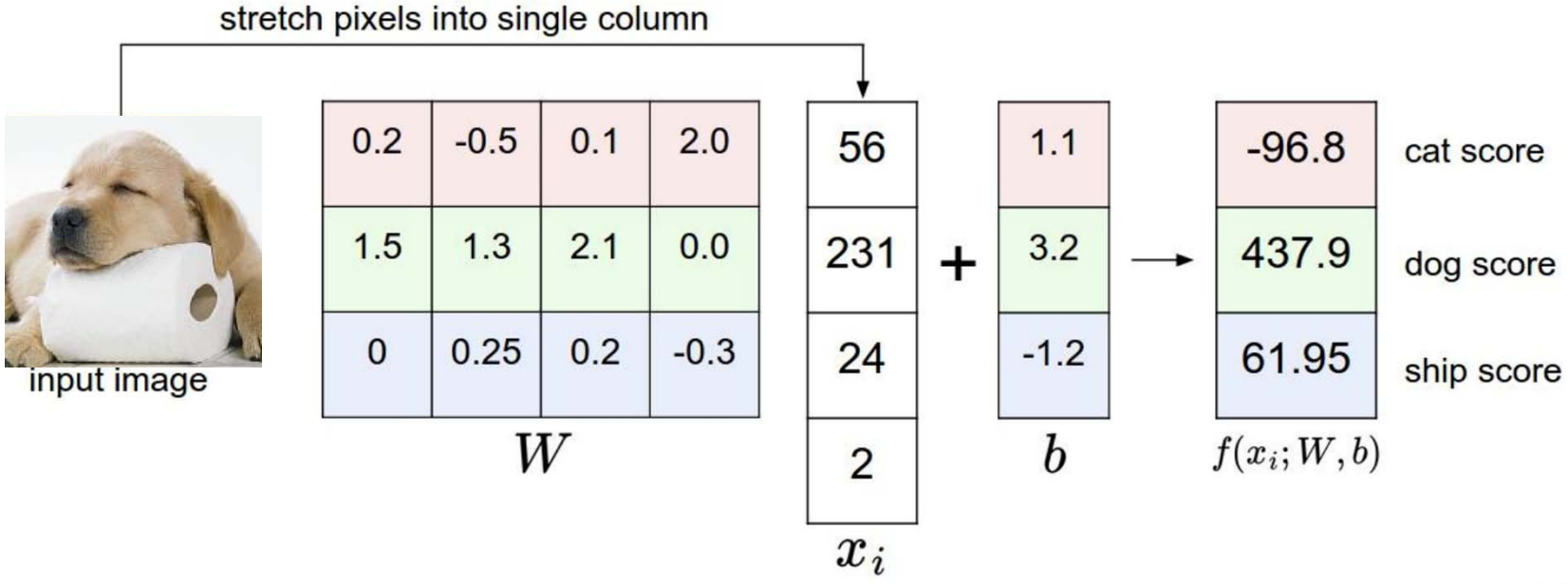
"bias vector"

"parameters"

Support Vector Machine

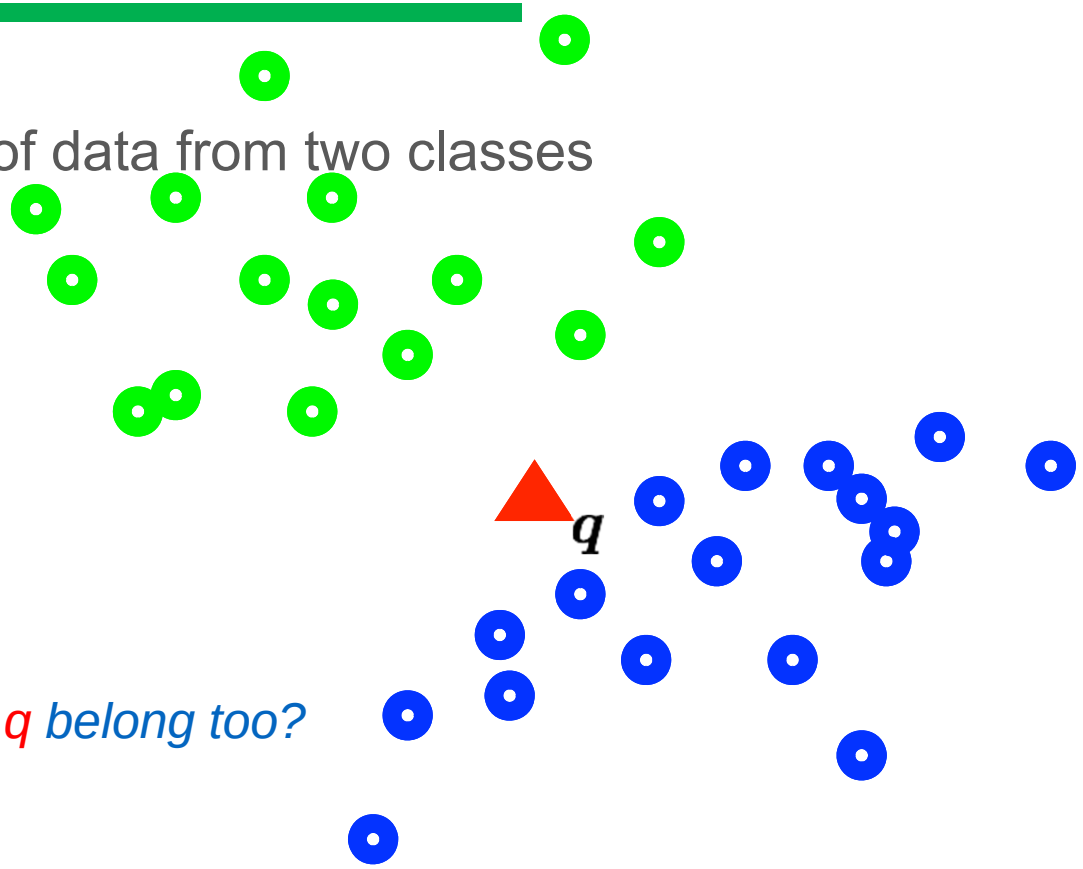
- Linear Classifier

Example with an image with 4 pixels, and 3 classes (cat/dog/ship)



Support Vector Machine

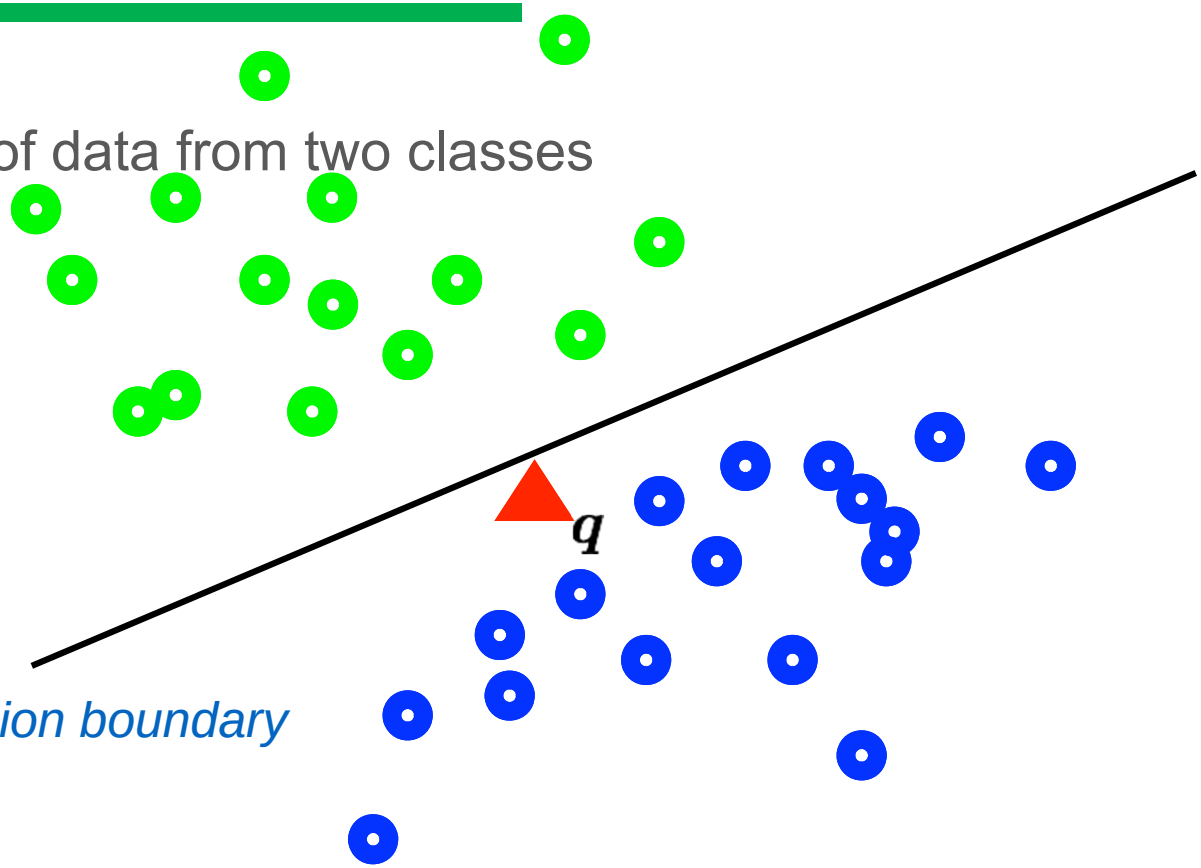
- Distribution of data from two classes



Which class does q belong too?

Support Vector Machine

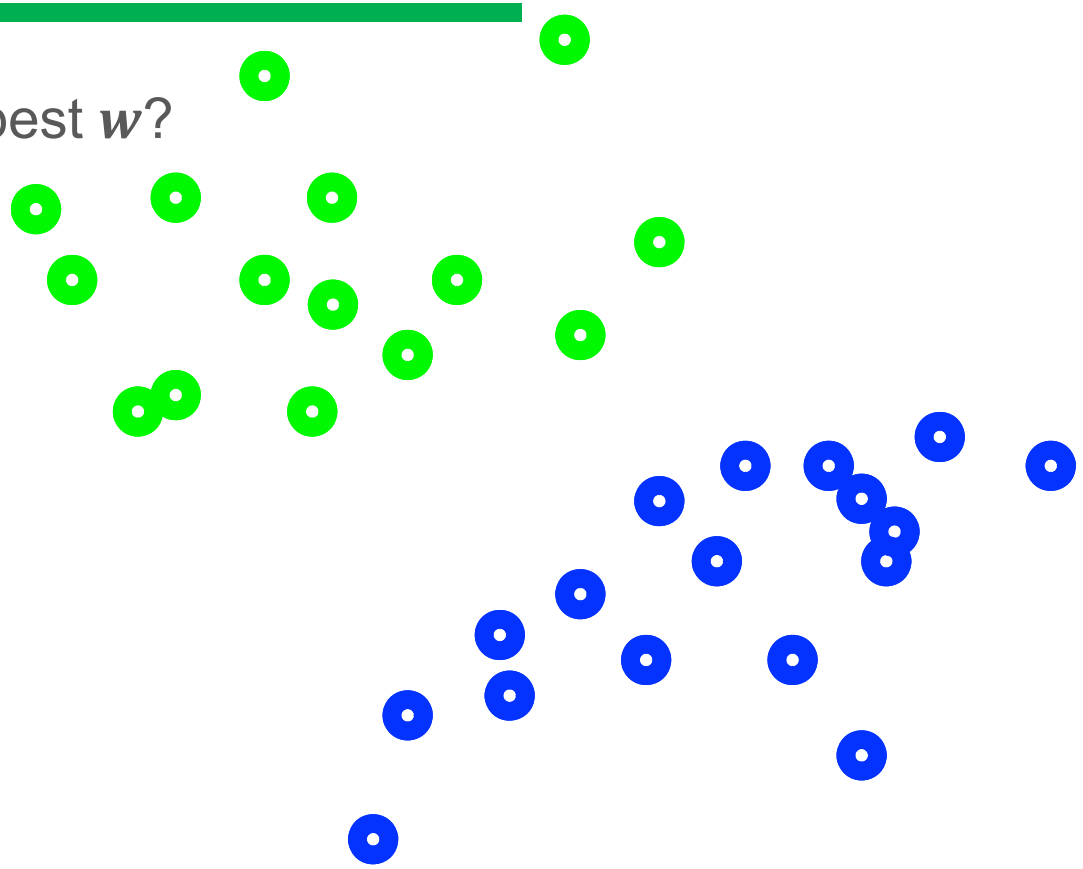
- Distribution of data from two classes



Learn the decision boundary

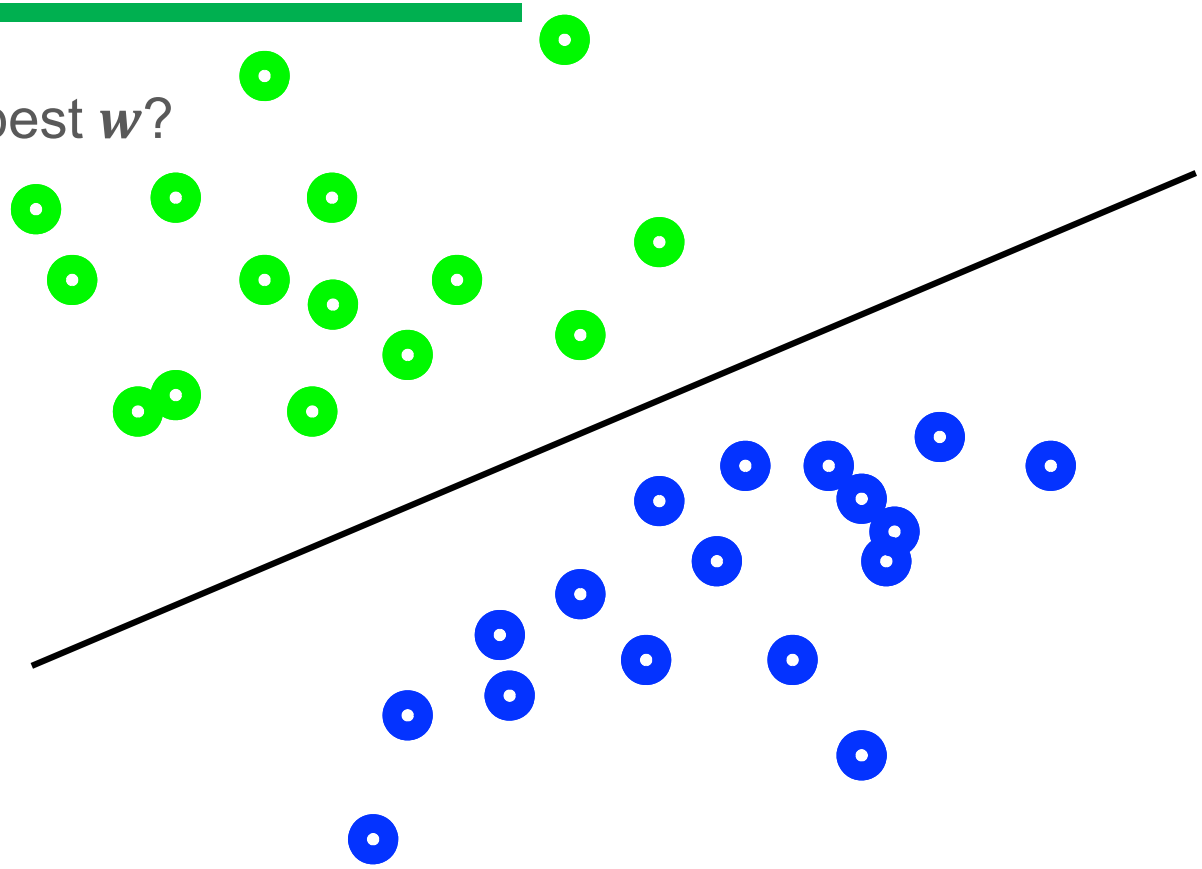
Support Vector Machine

- What's the best w?



Support Vector Machine

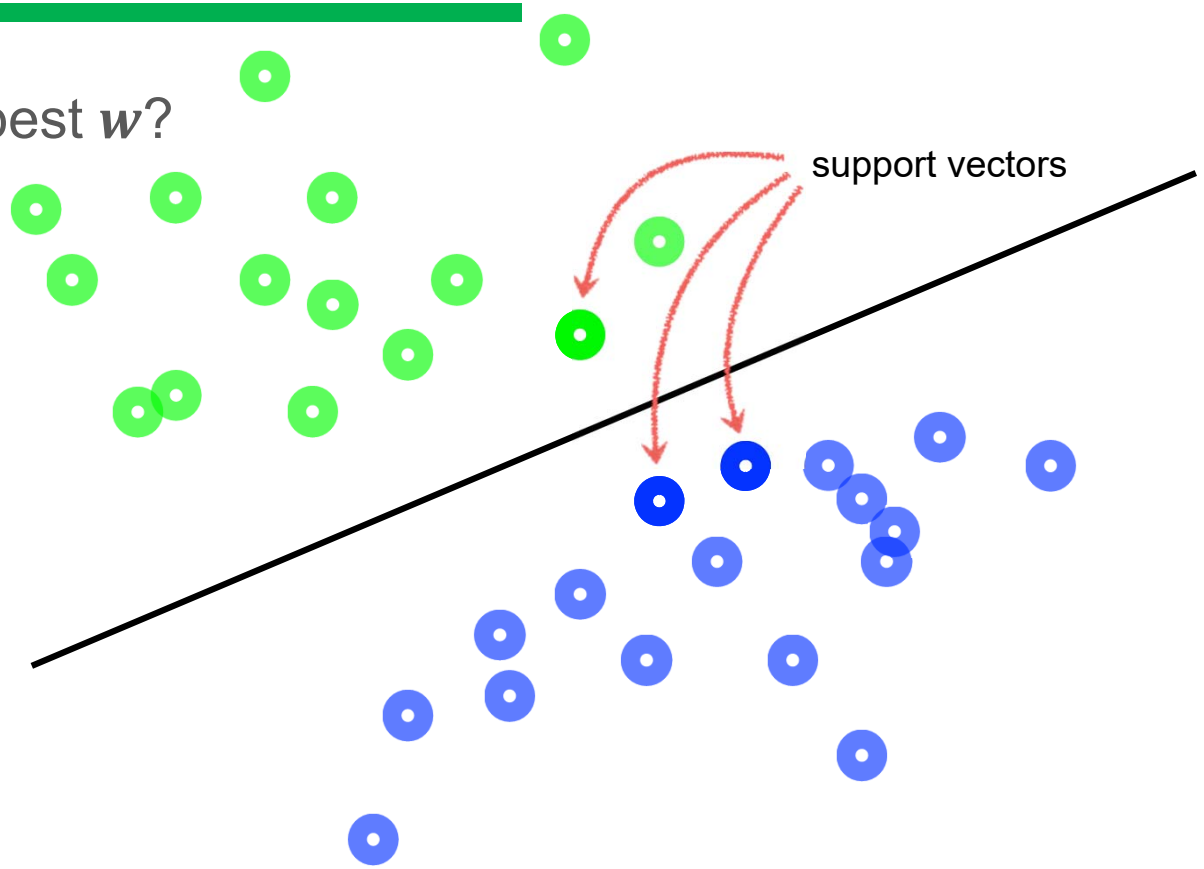
- What's the best w?



Maximum Margin solution:
most stable to perturbations of data

Support Vector Machine

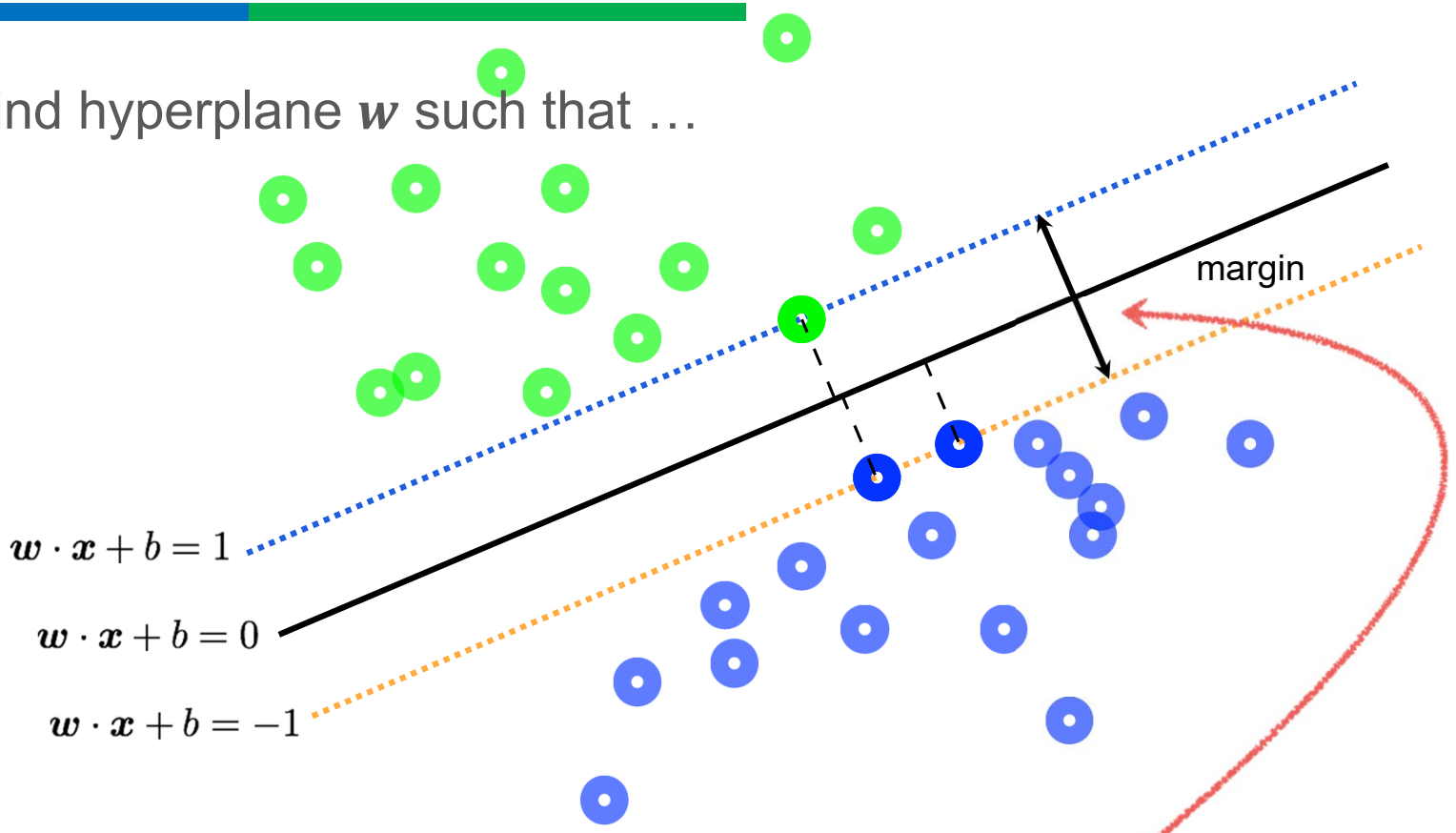
- What's the best w?



Want a hyperplane that is far away from 'inner points'

Support Vector Machine

- Find hyperplane $\boldsymbol{w}$ such that …



the gap between parallel hyperplanes $\frac{2}{\|\boldsymbol{w}\|}$ is maximized

Support Vector Machine

- Can be formulated as a maximization problem

$$\max_{\boldsymbol{w}} \frac{2}{\|\boldsymbol{w}\|}$$

$$\text{subject to } \boldsymbol{w} \cdot \boldsymbol{x}_i + b \begin{matrix} \geq +1 & \text{if } y_i = +1 \\ \leq -1 & \text{if } y_i = -1 \end{matrix} \text{ for } i = 1, \ldots, N$$



label of the data point

Support Vector Machine

- ‘Primal formulation’ of a linear SVM

$$\min_{\boldsymbol{w}} \|\boldsymbol{w}\| \longrightarrow J(\boldsymbol{\omega}) = \frac{|\boldsymbol{\omega}|^2}{2}$$

Objective Function

$$\text{subject to } y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) \geq 1 \text{ for } i = 1, \ldots, N$$

Constraints

This is a convex quadratic programming (QP) problem

(a unique solution exists)

Support Vector Machine

- Lagrange multipliers

$$L(\omega, d; \lambda_1, \lambda_2, \cdots, \lambda_N) = J(\omega) - \sum_i \lambda_i (y_i(\omega \cdot x_i + b) - 1)$$

$$= \frac{|\omega|^2}{2} - \sum_i \lambda_i (y_i(\omega \cdot x_i + b) - 1)$$

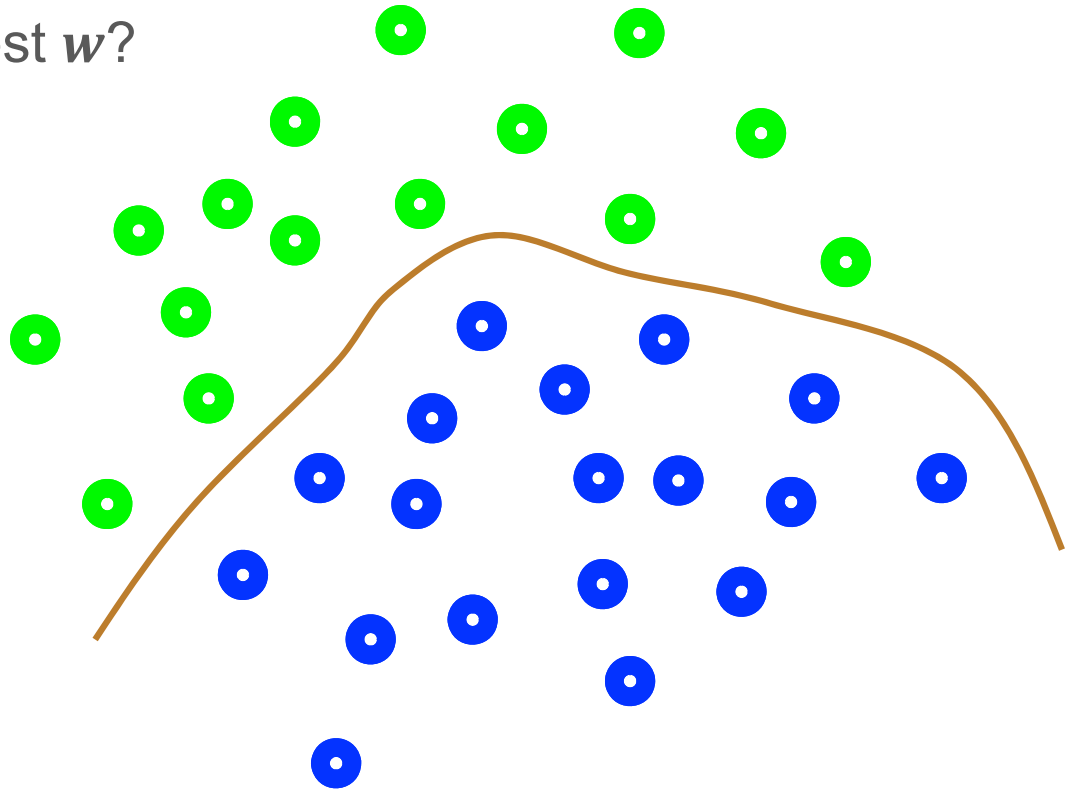
$$\sum_i \lambda_i y_i = 0, \lambda_i \geq 0\,, \quad \tilde{L}(\lambda) = \sum_i \lambda_i - \frac{1}{2} \sum_j \sum_j \lambda_i \lambda_j y_i y_j (x_j \cdot x_i)$$

Discriminative Function

$$y_t = sign(\sum_i \lambda_i y_i (x_i \cdot x_t) + \frac{1}{n} \sum_i (y_i - \sum_j \lambda_j y_i y_j (x_j \cdot x_i)))$$

Kernel Trick

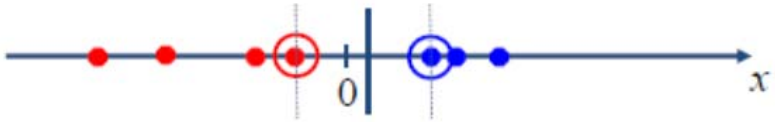
- What's the best w?



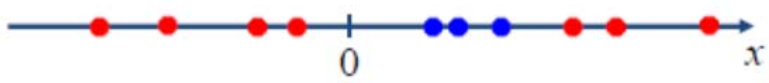
Intuitively, we should use non-linear concept

Kernel Trick

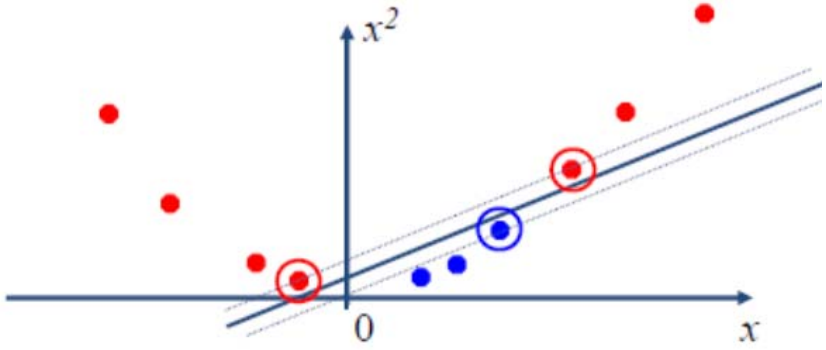
- Datasets that are linearly separable work out great



- But, what if the dataset is just too hard?

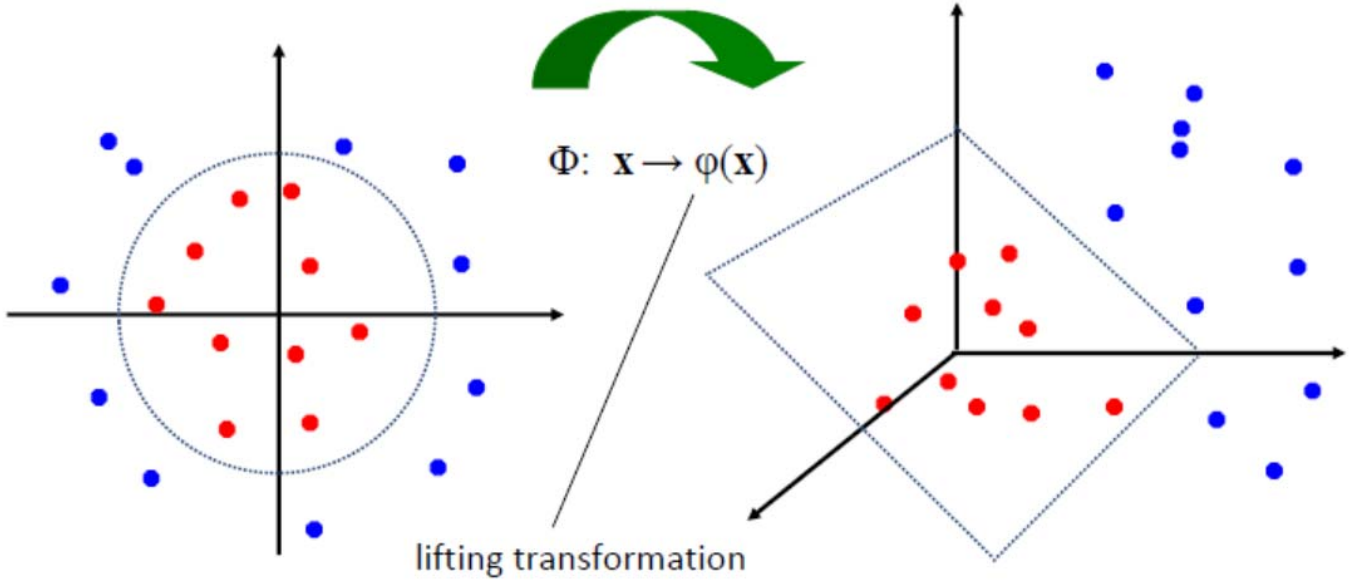


- We can map it to a higher-dimensional space



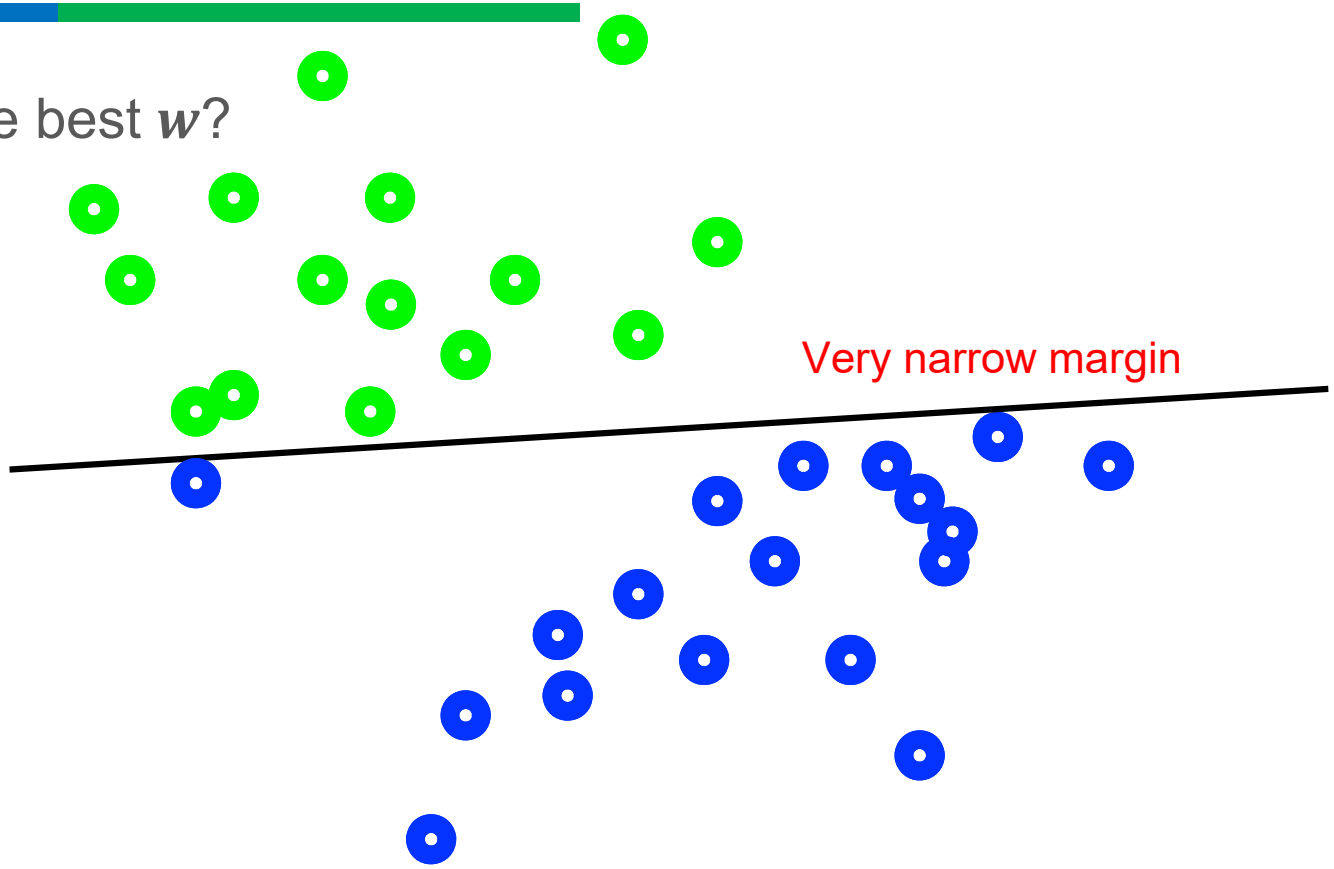
Kernel Trick

- General idea
  - The original input space can always be mapped to some higher-dimensional feature space where the training set is separable



‘soft’ margin

- What’s the best w?



‘soft’ margin

- Separating cats and dogs





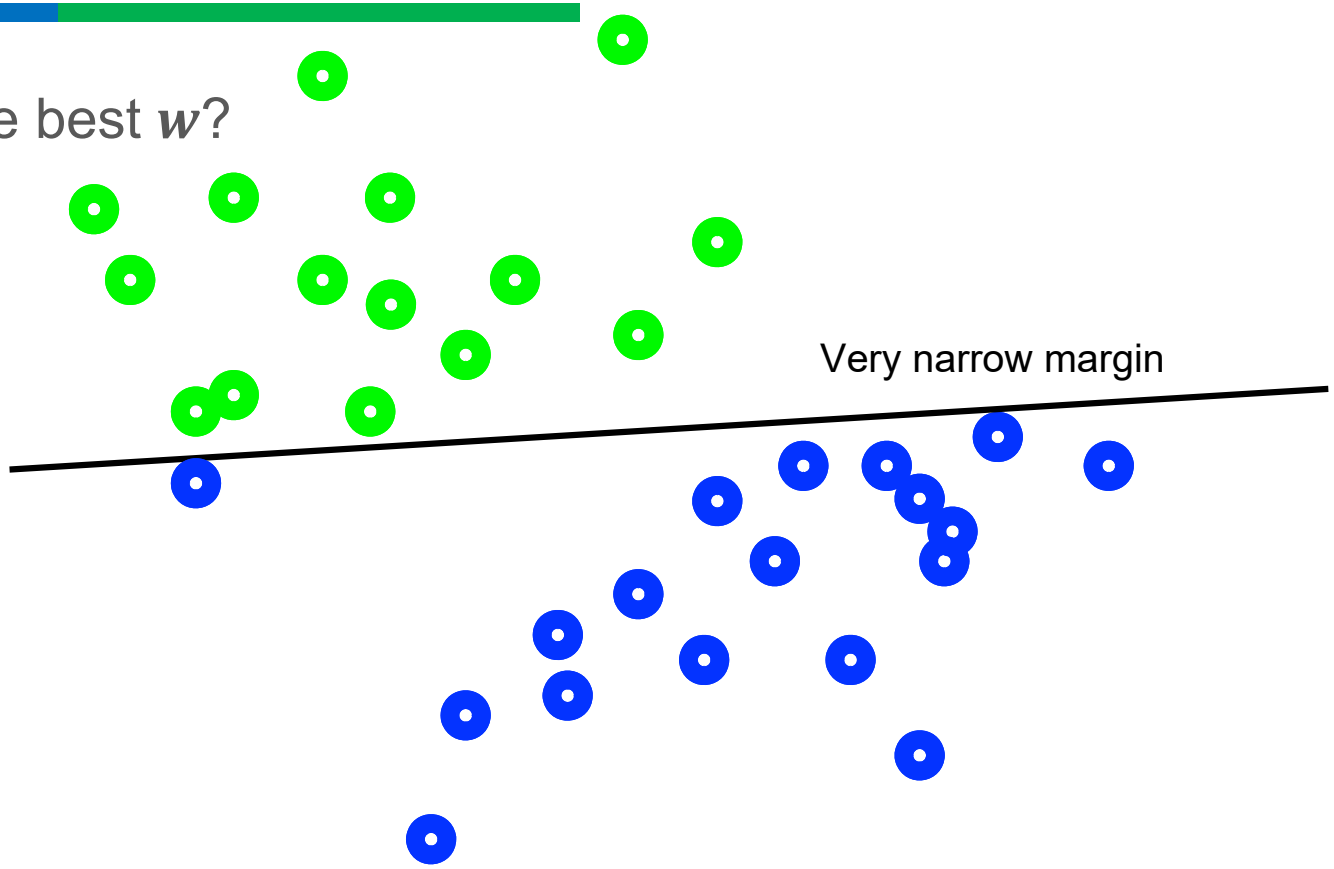
Very narrow margin





‘soft’ margin

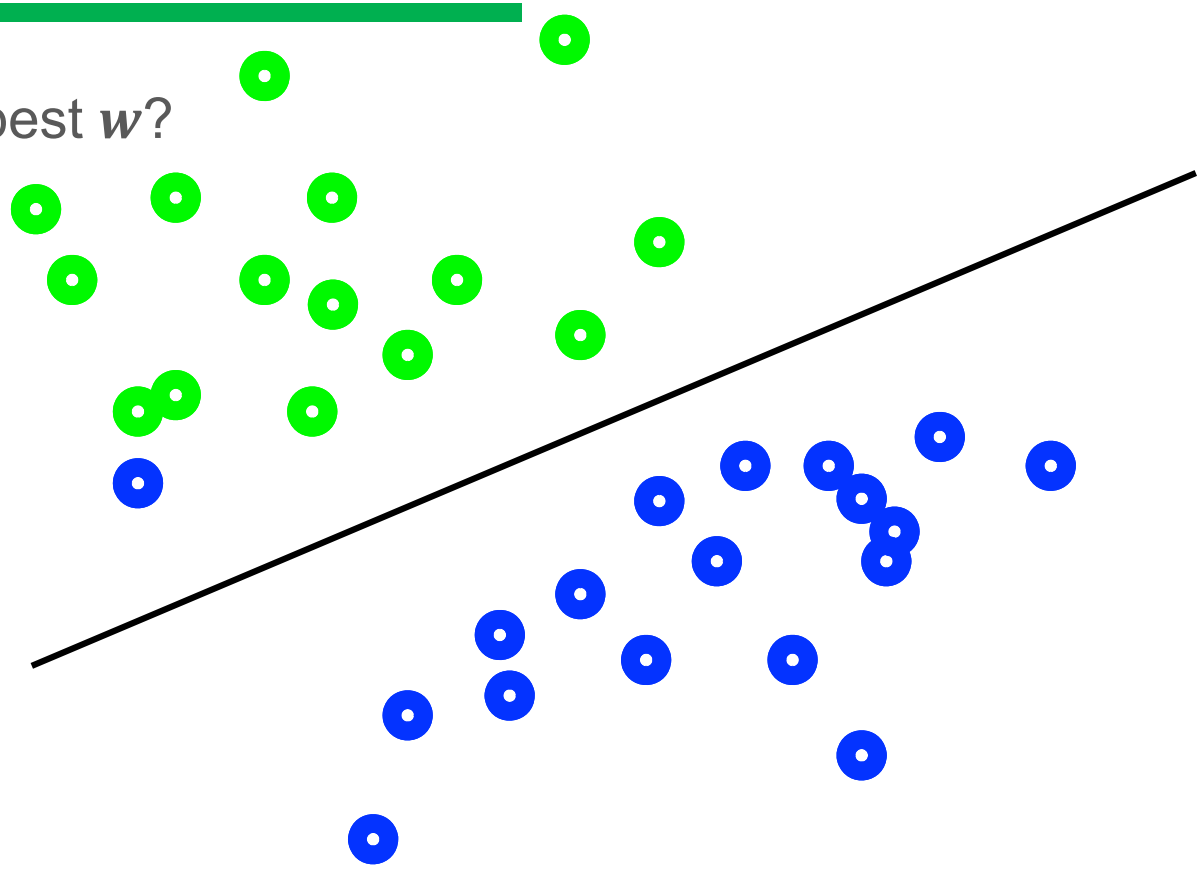
- What’s the best $\boldsymbol{w}$?



Intuitively, we should allow for some misclassification
if we can get more robust classification

‘soft’ margin

- What’s the best w?



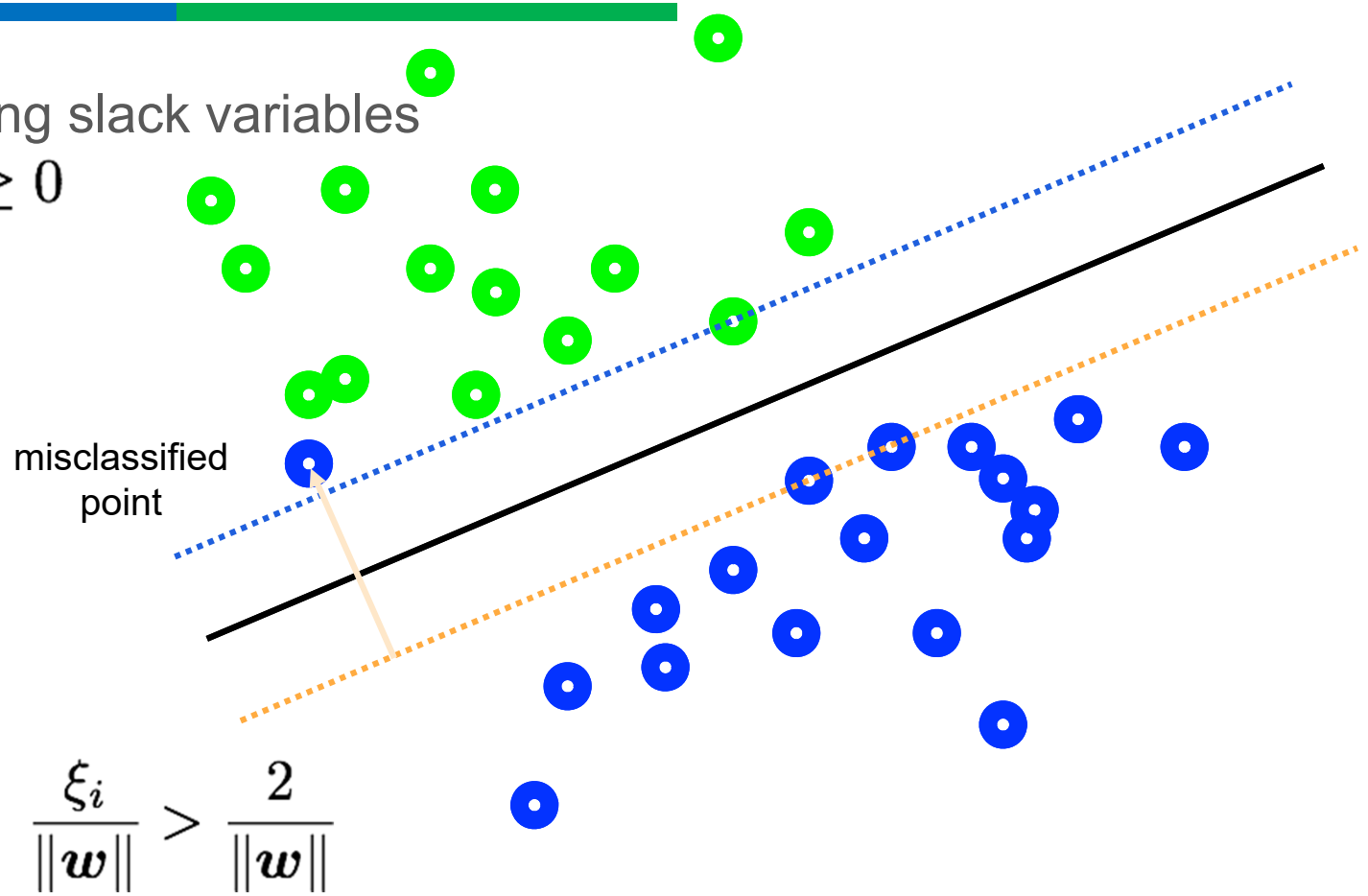
Trade-off between the MARGIN and the MISTAKES

(might be a better solution)

‘soft’ margin

- Adding slack variables

$$\xi_i \geq 0$$



$$\frac{\xi_i}{\|\boldsymbol{w}\|} > \frac{2}{\|\boldsymbol{w}\|}$$

‘soft’ margin

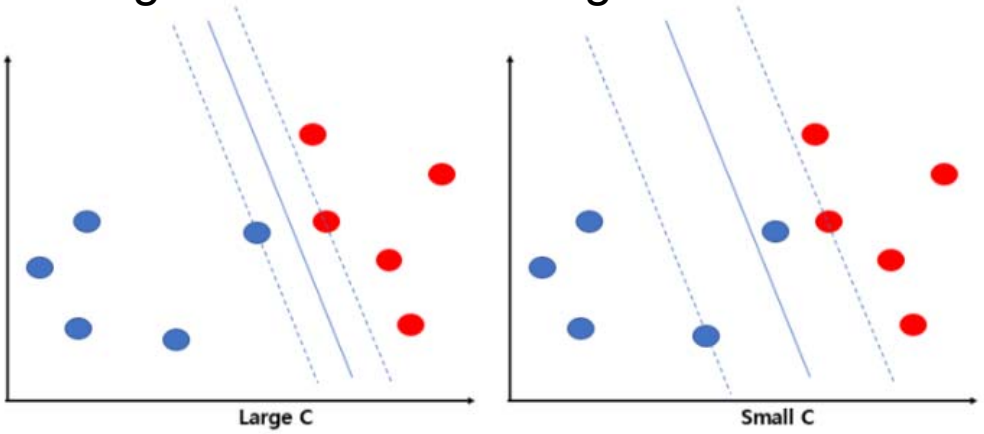
objective

$$\min_{\boldsymbol{w},\boldsymbol{\xi}} \|\boldsymbol{w}\|^2 + C\sum_i \xi_i$$

subject to

$$y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1 - \xi_i \quad \text{for} \quad i = 1, \ldots, N$$

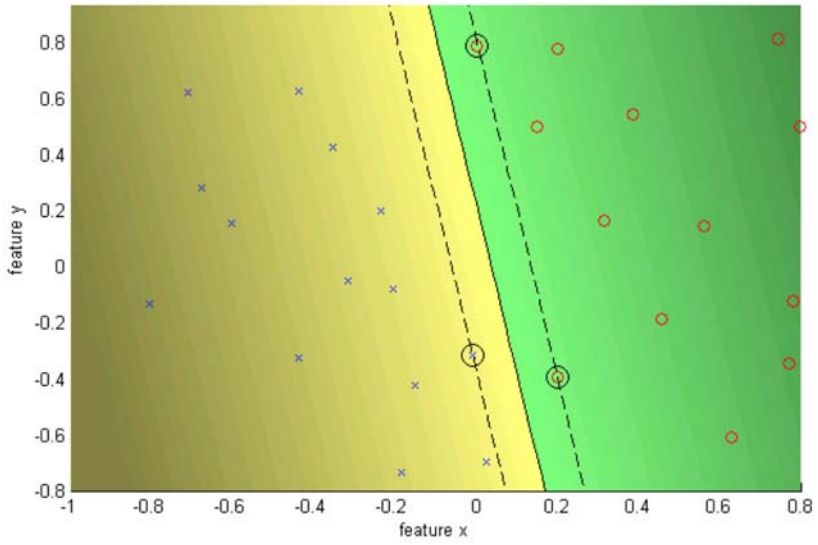
The slack variable allows for mistakes,
as long as the inverse margin is minimized.



‘soft’ margin

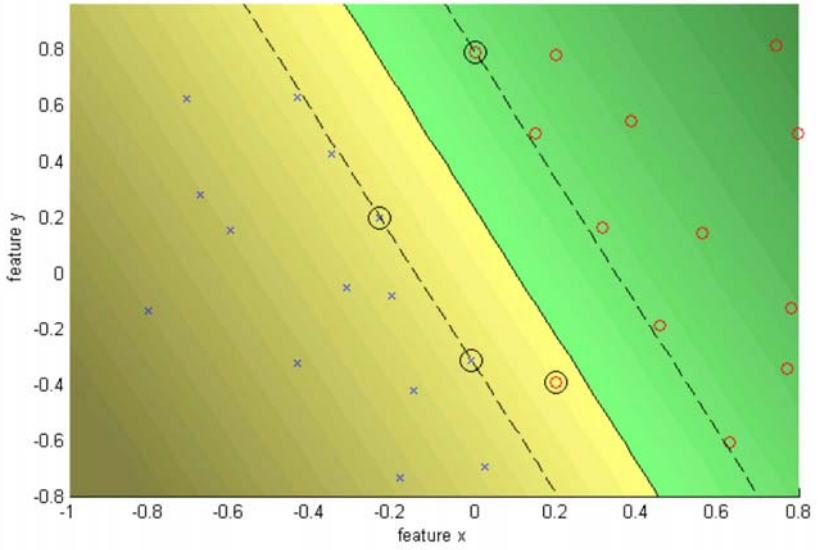
https://opencv24-python-tutorials.readthedocs.io/en/latest/py_tutorials/py_ml/py_svm/py_svm_index.html

C = Infinity hard margin





C = 10 soft margin





Any Questions?

Thanks!